

Programación modular

Juan Arnedo Moreno

Programación básica (ASX)
Programación (DAM)
Programación (DAW)

Índice

Introducción	5
Resultados de aprendizaje	7
1 Descomposición de problemas	9
1.1 Diseño descendente	9
1.1.1 Ejemplos de aplicación de diseño descendente	11
1.1.2 Reutilización de subproblemas resueltos	15
1.1.3 Aplicación correcta del diseño descendente	17
1.2 Diseño descendente aplicado a la creación de programas . . .	17
1.2.1 Declaración de métodos	18
1.2.2 Cambios en el método principal al declarar otros métodos 1.2.3	20
Accesibilidad de variables en una clase	21
1.2.4 Codificación de métodos	22
1.2.5 Invocación de métodos	23
1.2.6 Inicialización diferida de variables . . .	25
1.3 Un ejemplo más complejo . .	27
1.3.1 Descomposición de problema .	28
1.3.2 Esqueleto de la clase	31
1.3.3 Implementación del tercer nivel de descomposición .	32
1.3.4 Implementación del segundo nivel de descomposición .	34
1.3.5 Implementación del primer nivel de descomposición .	35
1.3.6 Implementación del problema general . .	36
1.3.7 Mejoras sobre la solución final	37
1.4 Solución de los retos propuestos	39
2 Parametrización de métodos	41
2.1 Parámetros de entrada	41
2.1.1 Motivación: definición de problemas similares	42
2.1.2 Declaración y uso de métodos con parámetros de entrada	44
2.1.3 Manipulación de los parámetros de entrada	47
2.2 Parámetros de salida	49
2.2.1 Motivación: Definición de problemas que generan un resultado concreto	49
2.2.2 Declaración y uso de métodos con un parámetro de salida	50
2.3 Cuando declarar parámetros de entrada o salida	53
2.3.1 Mejorando la legibilidad de código con parámetros . .	53
2.3.2 El principio de ocultación / encapsulación .	54
2.3.3 Ejemplos de métodos parametrizados . .	55
2.4 Un ejemplo de diseño descendente con métodos parametrizados . .	58
2.4.1 Descomposición de problema .	59
2.4.2 Implementación del segundo nivel de descomposición .	61
2.4.3 Implementación del primer nivel de descomposición .	62
2.4.4 Implementación del problema general . .	66

2.4.5 Implementación final	67
2.5 Soluciones a los retos propuestos	69

Introducción

Una vez conoce los tres tipos de estructura de control (secuencial, selección y repetición), ya dispone de todas las herramientas básicas para poder implementar algoritmos dentro de sus programas de ordenador, usando la técnica de la programación estructurada. Su aplicación correcta le permite generar un código pulido y ordenado, relativamente fácil de entender por una tercera persona. Sin embargo, el punto realmente más importante a la hora de llevar a cabo un programa de ordenador es la definición de su algoritmo. Si ya de entrada no diseña un algoritmo correcto, el programa nunca funcionará correctamente, independientemente de que se usen correctamente los principios de la programación modular o que el código fuente no tenga errores de sintaxis.

Para programas sencillos, el diseño de algoritmos se puede realizar de forma ad hoc, pensando los diferentes pasos ordenados a seguir en lenguaje natural, que después pueden ser traducidos a instrucciones del código fuente. Ahora bien, para programas complejos, en los que existen muchos pasos interrelacionados, puede ser fácil perderse, o crear una descripción enrevesada o difícil de seguir. Hay tantos detalles que le resultará imposible tenerlos todos presentes a la vez en la cabeza. El resultado es que resulta mucho más fácil equivocarse o, incluso, una vez se sabe que existe un error en el diseño, intentar averiguar dónde se encuentra éste. Por tanto, es útil disponer de una metodología de trabajo que eche una mano en este aspecto. En esta unidad se presentará una metodología bastante popular y útil a la hora de diseñar algoritmos complejos: el diseño descendente. En pocas palabras, éste se basa en la estrategia, aplicable a cualquier campo y no sólo a la programación, de dividir problemas complejos en un conjunto de problemas más simples y fáciles de atacar y entender.

En el apartado “Descomposición de problemas” se exponen los principios del diseño descendente aplicado en el contexto de la creación de algoritmos. Para poder llevarlo a cabo, se presenta un nuevo tipo de blog de instrucciones que puede usar dentro de sus programas, los métodos. Si bien éstos ya habían sido introducidos con anterioridad, sólo se habían visto desde el punto de vista de su invocación por manipular cadenas de texto. Aquí verá con un poco más de detalle cómo funcionan realmente, su utilidad y cómo puede definir otros nuevos, inventados por usted, en sus programas. Sin embargo, de momento este apartado sólo ofrece una visión general sobre cómo su existencia está vinculada a la aplicación del diseño descendente de algoritmos, sin ver todos sus detalles.

Los detalles más complejos de la declaración y uso de métodos se detallan en el apartado “Parametrización de métodos”. Aquí se presenta su utilidad como mecanismo para manipular y producir datos, de modo que su uso resulte mucho más versátil y puedan ser invocados con resultados similares a los descritos por operar con cadenas de texto. Esto permite crear una recopilación de herramientas reusables dentro de sus programas para transformar información, de forma que se evite tener que escribir código repetido.

Debe tener en cuenta que la declaración de métodos y su invocación es una herramienta usada con asiduidad a los programas de cualquier lenguaje de programación, por lo que comprenderla y saber cuándo usarla es de capital importancia para un programador. En el mundo real, es complicado realizar programas complejos que no incluyan métodos. Por tanto, conviene que a partir de ahora siempre que realice un programa intente aplicar los mecanismos explicados en este módulo para dividir el código en diferentes métodos, y si puede ser, reaprovecharlos en diferentes lugares dentro del programa.

Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Analiza los conceptos relacionados con la programación modular
2. Analiza las ventajas y la necesidad de la programación modular
3. Aplica el concepto de análisis descendente en la elaboración de programas
4. Modulariza correctamente los programas realizados.
5. Realiza correctamente las llamadas a funciones y su parametrización.
6. Tiene en cuenta el ámbito de las variables en las llamadas a las funciones

1. Descomposición de problemas

Al igual que con la inmensa mayoría de tareas con cierto grado de complejidad dentro del mundo real, la creación de un programa requiere un paso previo donde es necesario reflexionar sobre qué es exactamente lo que desea hacer y cómo alcanzará su meta. Es muy poco recomendable afrontar esta tarea ya asiendo directamente frente al ordenador, abriendo el entorno de trabajo y empezando a escribir líneas de código. Esta opción sólo es realmente factible cuando dispone de una cierta experiencia programante y encuentra que el problema que debe resolver, o ya lo ha tratado con anterioridad, o se parece mucho a otro que ya ha resuelto. Pero cuando se enfrenta con un problema nuevo es imprescindible una etapa en la que estudiar el problema, los datos que desea tratar exactamente y las tareas que debe llevar a cabo el ordenador para hacerlo (o sea, el algoritmo del programa).

Desgraciadamente, la capacidad de los humanos para captar problemas complejos es limitada, ya que, por lo general, sólo somos capaces de mantener una visión simultánea de unos pocos elementos. A este hecho hay que añadir que la toma de una decisión sobre qué ocurre es necesario llevar a cabo dentro de la descripción de un proceso siempre tiene implicaciones sobre futuros pasos. Por tanto, cuando el proceso a realizar es largo o se basa en la manipulación de muchos elementos diferentes, es muy fácil, no ya simplemente equivocarse, sino tan sólo saber por dónde empezar.

Una vez llegados a este punto, resulta evidente que resultaría útil disponer de alguna estrategia que permita hacer frente a la resolución de problemas con diferentes grados de complejidad. Una de las más populares en todos los campos, y que seguro usa a menudo en su día a día, quizás sin darse cuenta, es considerar que un problema complejo en realidad no es más que la agregación de un conjunto de problemas más simples, cada uno de ellos más fáciles de resolver. Por tanto, si usted es capaz de entender y resolver todo este conjunto de problemas simples, también podrá ser capaces de resolver el problema complejo.

En consecuencia, y partiendo de esta premisa, el primer paso para poder llevar a cabo una tarea compleja será encontrar cómo descomponerla en otras más simples, que entonces se irán resolviendo uno por uno.



Como dice la frase que se atribuye a Filipo II de Macedonia: "Divide te impera" (divide y vencerás). Fuente: Tilemahos Efthimiadis

1.1 Diseño descendente

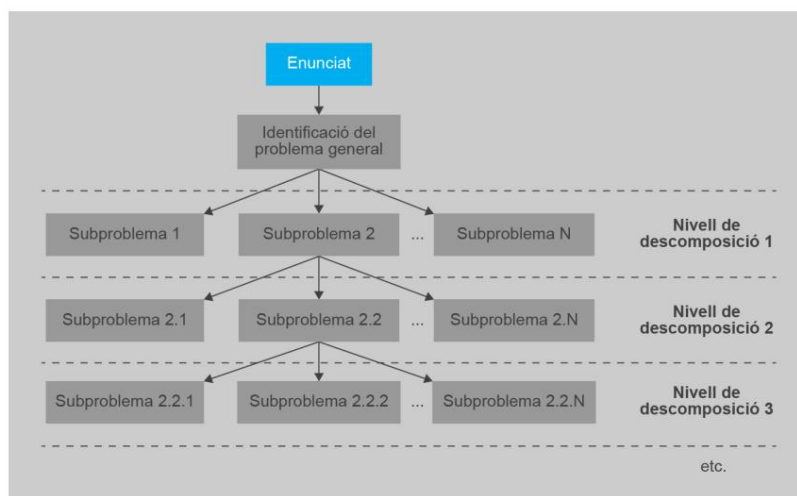
Existen dos estrategias básicas para resolver la descomposición de un problema: el diseño descendente y el ascendente. Este apartado se centra en la primera, en ser la más utilizada por norma general, y la más fácil de aplicar a la hora de crear programas del nivel que abarcan estos materiales.

En el contexto de la programación, el diseño ascendente normalmente se aplica en el campo de la orientación a objetos.

El diseño descendente (top-down, en inglés) es la técnica basada en partir de un problema general y dividirlo en problemas más simples, denominados subproblemas. De todos ellos, los considerados aún demasiado complejos se vuelven a dividir en nuevos subproblemas. Se llama descendente porque en base al problema grande se pasa a problemas más pequeños a los que dará solución individualmente.

El esquema de aplicación de esta estrategia se muestra en la figura 1.1, en la que se ve la razón del nombre descendente, y se aprecia cómo, partiendo de la definición del problema general, extraída de tarea final que desea alcanzar, se crea una estructura jerárquica de subproblemas en distintos niveles. El número de niveles alcanzados dependerá de la complejidad del problema general. Para problemas no demasiado complejos, será suficiente con uno o dos niveles, pero para resolver problemas muy complejos puede ser necesario un gran número de sucesivas descomposiciones. También merece la pena remarcar que, aunque es recomendable que la complejidad de los subproblemas de un mismo nivel sea aproximadamente equivalente, puede haber que resueltos completamente en menos niveles que en otros.

Figura 1. 1. Esquema de aplicación de diseño descendente, de acuerdo a los niveles de descomposición del problema



Un punto importante a tener en cuenta al aplicar esta descomposición es que cada uno de los subproblemas no se genera arbitrariamente, sino que se plantea como objetivo parcial, con entidad propia, para resolver su problema de nivel superior. Una vez alcanzados todos estos objetivos parciales, se considera resuelto el total.

Los objetivos finales de aplicar esta estrategia son:

- Establecer una relación sencilla entre problemas planteados y el conjunto de tareas a realizar para resolverlos.
- Establecer más fácilmente los pasos para solventar un problema.
- Hacer más fácil entender estos pasos.
- Limitar los efectos de la interdependencia que tiene un conjunto de pasos sobre otro conjunto.

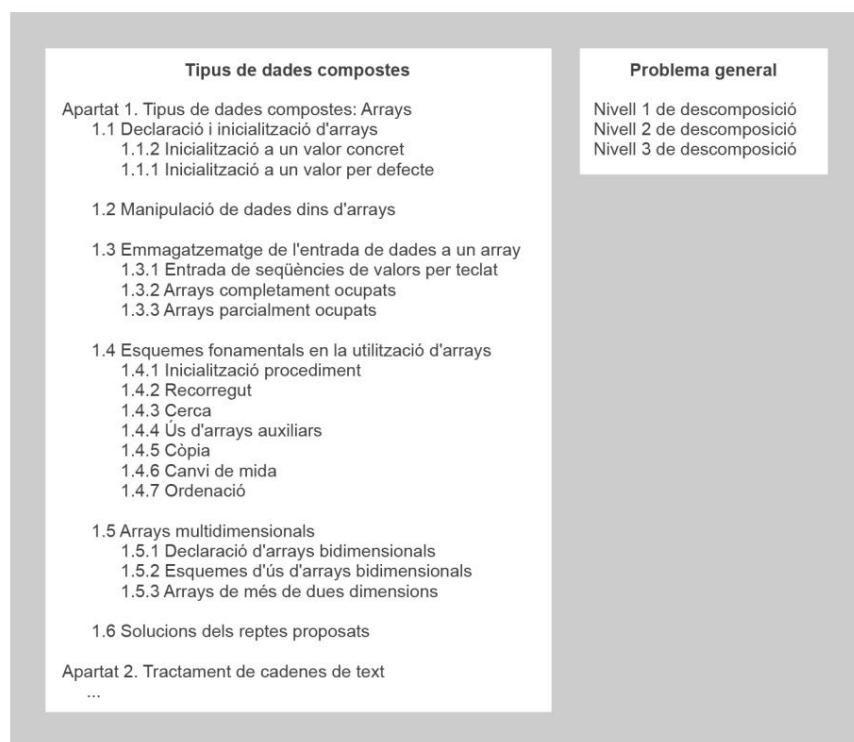
1.1.1 Ejemplos de aplicación de diseño descendente

Como siempre, la mejor forma de ver la aplicación de diseño descendente es mediante ejemplos. Para empezar, éstos se centrarán en la resolución de problemas de ámbito general, que permitan ver con claridad el proceso sea cual sea el contexto, antes de entrar en el caso concreto de la generación de un programa.

Una unidad formativa

Un caso bastante directo de aplicación de diseño descendente es la escritura de un documento de cierta complejidad, como por ejemplo, un libro o unidad formativa del IOC. Se trata de un caso directo puesto que la estructura de un documento de este tipo es evidentemente jerárquica, basada en capítulos, secciones, subsecciones, etc. Así pues, justo delante de usted ahora mismo tiene el resultado directo de aplicar diseño descendente sobre un problema.

Figura 1. 2. Descomposición de una Unidad Formativa siguiendo el diseño descendente



En un caso como éste, cuando se plantea que es necesario hacer una unidad formativa, aunque se parte ya de unas directrices o idea general de lo que se quiere explicar (por ejemplo, unos objetivos de aprendizaje), estará de acuerdo en que no sería muy sensato sentarse ya inmediatamente ante un procesador de texto y ponerse a escribir, atacando frontalmente el problema. Esta aproximación normalmente lleva a no saber exactamente por dónde empezar, o simplemente poner fin a un texto incomprensible, con incluso explicaciones repetidas. Es más eficiente empezar con una etapa de diseño en la que se establezca un primer nivel de objetivos parciales en la redacción: una división inicial en apartados. Este primer nivel de descomposición seguro

que todavía será demasiado genérico, pero ya ha dividido el problema inicial en otros más pequeños.

Una vez llegados a este punto, por cada apartado, se van realizando sucesivas divisiones en secciones, subsecciones, etc. partiendo de conceptos más generales que quieren explicarse hacia conceptos más específicos. Una vez se considera que se ha llegado a un concepto lo suficientemente específico como para poder ser explicado de forma autocontenida y de forma relativamente fácil de entender por el lector, ya no es necesario descomponer más. Evidentemente, una vez tratados todos los apartados, es posible encontrarse con que algunos apartados o secciones están divididos en más subapartados que otros. Esto no es problema. Ahora bien, lo que sí debe ser cierto siempre es que cada subapartado se corresponda a una temática concreta, con una entidad y sentido propio (de acuerdo a su título), y nunca se trata de un “cajón de sastre” en el que se explican muchas cosas diferentes poco relacionadas entre ellas. Por ejemplo, la figura 1.2 muestra la descomposición de una unidad de acuerdo a estos criterios.

Como puede verse, en este caso, el resultado de aplicar la descomposición le da como resultado el índice de la unidad. Estará de acuerdo en que es mucho más sencillo editar un documento partiendo de un índice preestablecido, con nombres de secciones autoexplicativos sobre lo que deben tratar, que actuando de forma improvisada. Adicionalmente, este proceso de descomposición alcanza otro hito muy importante que va más allá de facilitar la etapa de redacción del texto. El documento resultante resulta también mucho más fácil de seguir y entender por parte de los futuros lectores.

Una receta de cocina

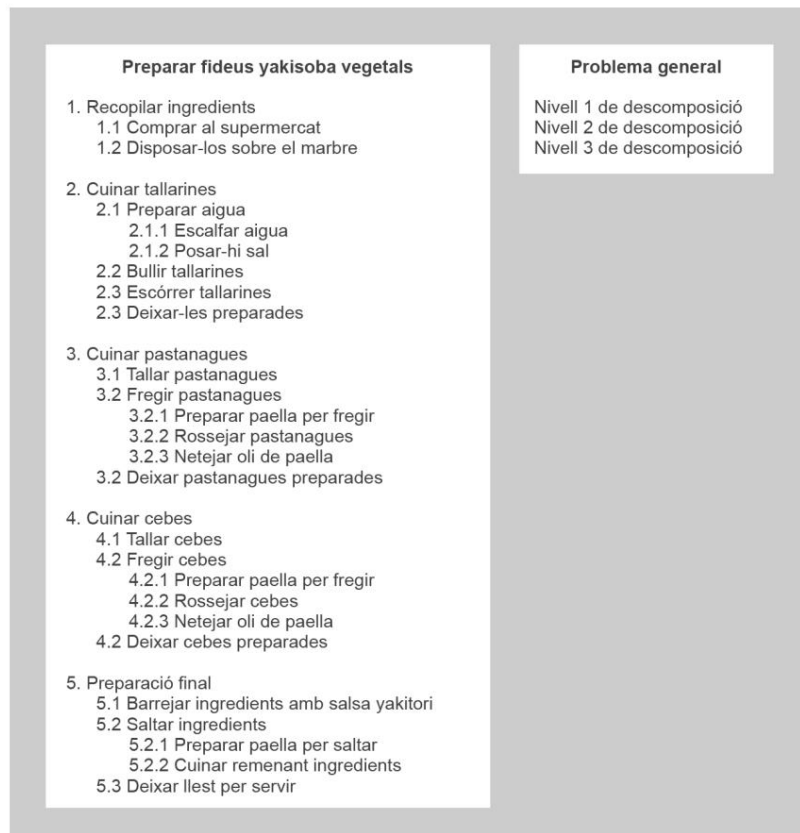
Si bien el ejemplo del documento de texto es prácticamente de los más directos que hay para ilustrar cómo la división de un problema en otros más pequeños es de gran ayuda, existe un pequeño detalle a tener en cuenta. La mayoría de documentos de texto, y es el caso de las unidades formativas, no describen algoritmos, que es al fin y al cabo lo que tendrá que hacer en un programa. Ahora bien, basta con que el texto describa una secuencia de pasos para llevar a cabo un hito para que ya se convierta en algoritmo. Por ejemplo, un manual de instrucciones para montar un mueble o receta de cocina. En estos casos, para su redacción, el proceso general se divide igualmente en apartados y secciones que se corresponden a tareas individuales y concretas dentro del proceso general.



Una receta de cocina es un algoritmo, y se puede descomponer usando diseño descendente
(Fuente: jetalone)

Por tanto, lo que se propone hacer, a modo de ejemplo, es llevar a cabo un proceso de descomposición en subproblemas de una receta de cocina de fideos japoneses yakisoba () sin carne. En un caso como éste, el concepto de problema complejo es relativo, puesto que todo depende de las habilidades y conocimientos culinarios del lector. Para no simplificar demasiado el ejemplo, se supondrá que algunos aspectos como freír o saltar no se consideran tareas simples, y es necesario tener bien presente el proceso de preparación. La figura 1.3 muestra una propuesta de esquema de descomposición usando diseño descendente. El formato utilizado para establecer el nivel de descomposición será el mismo que en el ejemplo anterior. Estudie atentamente y dedique unos momentos a reflexionar si usted lo habría hecho de otra manera.

Figura 1. 3. Descomposición de la preparación de una receta de cocina



Dado que este ejemplo sí está expresando la descomposición de un proceso, existen algunos aspectos a tener en cuenta. Por un lado, note cómo la nomenclatura usada para identificar cada subproblema indica claramente qué se quiere alcanzar, de modo que tanto usted como un tercer observador puede tener una idea clara de lo que hay que resolver para llevar a cabo la tarea final. Esto es muy importante. Ahora bien, por otra parte, si bien con este identificador se sabe “qué” es necesario resolver, no se sabe “cómo” se resuelve cada subproblema. En ese sentido, se considera que un subproblema es una abstracción sobre parte del proceso completo. Definir el algoritmo que resuelve cada subproblema individual será ya la siguiente etapa.

Por ejemplo, una vez clara la descomposición, ya se podría decidir buscar el algoritmo para resolver el subproblema “Preparar aceite para freír”, que podría ser:

1. Coger botella de aceite de girasol.
2. Llenar sartén hasta un tercio.
3. Poner fuego al máximo.
4. Mientras el aceite no se caliente.
5. Esperar.

Nótese cómo, para resolver este punto, no es necesario saber absolutamente nada del resto del proceso general. Esto indica que cada subproblema del nivel más bajo plantea una serie de tareas totalmente autocontenidas. Este proceso

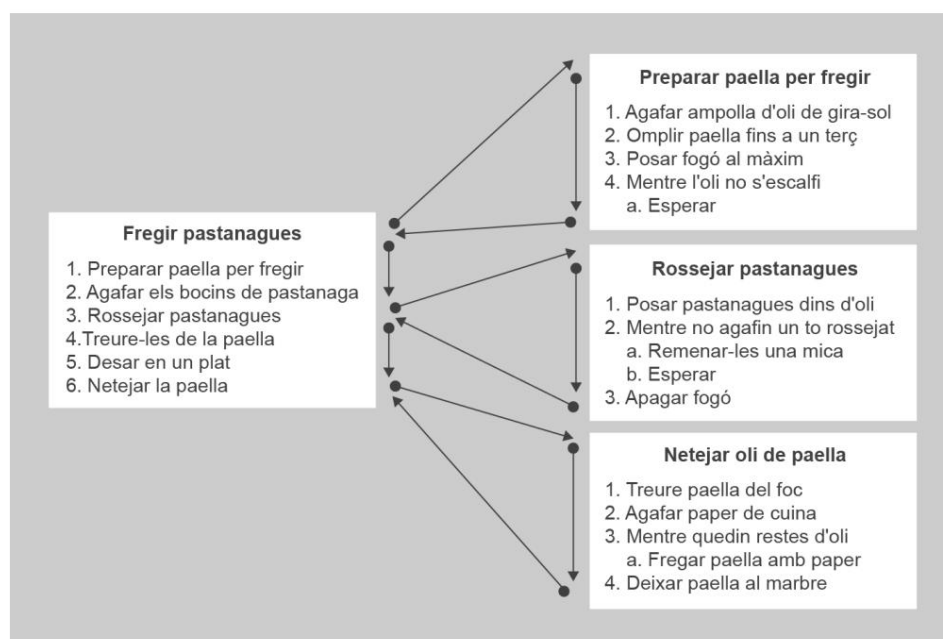
entonces se iría repitiendo por cada subproblema, normalmente comenzando resolviendo los problemas más sencillos (niveles inferiores), y yendo poco a poco resolviendo los más complejos (niveles superiores), hasta llegar al problema general, que es el de mayor nivel.

Este orden recomendado se debe a que durante este proceso, para resolver subproblemas de niveles superiores, es posible referirse a subproblemas de niveles inferiores. La mejor manera de ver esto es viendo cómo se resolvería, por ejemplo, el subproblema “Fregir zanahorias”:

1. Preparar aceite para freír.
2. Coger los pedazos de zanahoria.
3. Meer zanahorias.
4. Sacarlas de la sartén.
5. Guardar en un plato.
6. Limpiar la sartén.

Nótese cómo, para resolver este subproblema, en los puntos 1, 3 y 6 precisamente se está refiriendo a subproblemas de nivel inferior, que se consideran resueltos si seguimos el orden de resolución descrito. También note cómo, para resolver un subproblema, tanto se pueden usar subproblemas ya resueltos de nivel inferior como pasos adicionales que se consideren suficientemente simples. Por tanto, desde el punto de vista de orden de los pasos que se están siguiendo, el proceso seguiría a grandes rasgos el flujo de control de la figura 1.4.

Figura 1. 4. Flujo de control del algoritmo basado en diseño descendente para freír unas zanahorias



En esta figura, el punto más importante es ver cómo cada subproblema se considera una entidad estrictamente independiente y autocontenida dentro de todo el

proceso, al que se accede desde otro de nivel superior. Cuando se hace, sus pasos son seguidas de inicio a fin, y al terminar se continúa exactamente por dónde se había quedado en el nivel superior.

1.1.2 Reutilización de subproblemas resueltos

Los ejemplos de la unidad formativa o la receta de cocina, a simple vista, aparentan seguir un esquema muy similar. Los diferentes niveles siguen una ordenación secuencial (1, 1.1, 1.1.1...2, 2.1, etc.), de modo que, a la postre, los subproblemas se van resolviendo de forma ordenada y una vez resuelto el último subproblema, la tarea general está prácticamente finalizada. Esto encaja con el modelo estrictamente jerárquico del problema general descompuesto tal y como se ha expuesto inicialmente.

Sin embargo, la descomposición mediante diseño descendente permite hacer uso de una característica muy útil cuando se usa para diseñar algoritmos. Se trata de la posibilidad de buscar subproblemas idénticos, o al menos bastante similares, y reaprovechar su solución en más de un lugar dentro del problema general. Una vez que se han resuelto una vez, no tendría sentido volver a resolverlos de nuevo repetidas veces. Sobre esta circunstancia, por el momento se estudiará sólo el caso de subproblemas exactamente iguales.

Por ejemplo, si se fija en la descomposición de la receta de cocina, puede observar que hay subproblemas repetidos. Se trata de “Preparar sartén para freír” y “Limpiar sartén”. No sólo se han descrito ya de entrada con nombres idénticos, sino que, si se pare a pensar, los pasos que engloban también lo serán. Los elementos que se manipulan para su resolución (sartén y aceite) y la forma en que se hace esta manipulación son exactamente los mismos. Por tanto, una vez se han definido los pasos para resolverlo la primera vez, ya no es necesario volver a hacerlo.

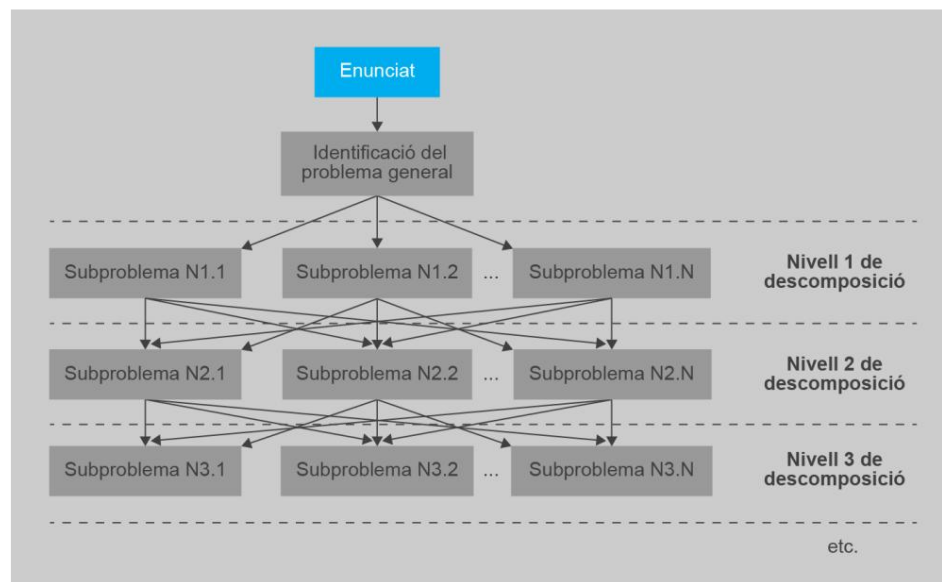
Tenga cuidado, ya que por considerar que dos soluciones son idénticas, los elementos que intervienen deben ser exactamente los mismos. Así pues, “Cortar cebollas” y “Cortar zanahorias” son ciertamente subproblemas muy parecidos, pero no realmente idénticos, ya que se manipulan elementos diferentes.

Una vez detectada esta característica del diseño descendente, es el momento de matizar la descripción de la descomposición en niveles de la figura 1.1. En realidad, el proceso es más parecido al descrito en la figura 1.5, que indica un cambio de planteamiento, ya que cualquier subproblema de un nivel dado puede ser parte de cualquier subproblema de un nivel superior. Para remarcar este hecho, en la figura cada subproblema no se enumera usando un índice asociado al subproblema de nivel superior al que pertenece, sino directamente de acuerdo al nivel al que pertenece. No hay exclusividad en la jerarquía.



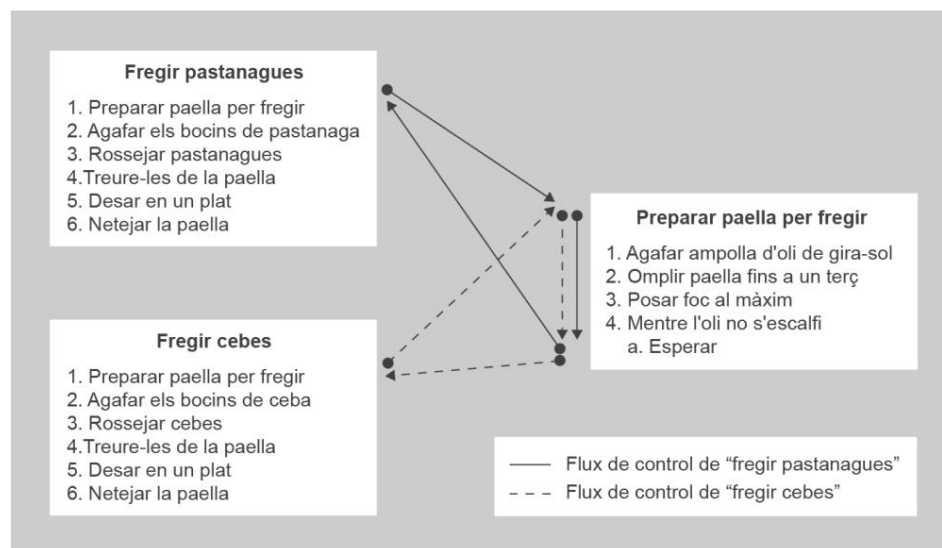
Reaprovechando subproblemas resueltos se ahorrará reinventar la rueda. Fuente: Derek Ramsey

Figura 1. 5. Esquema de aplicación de diseño descendente con subproblemas repetidos



Esta circunstancia también hace que, a la hora de considerar el flujo de control del algoritmo, éste adopte una forma especial. Para resolver dos tareas distintas se comparte un mismo conjunto de pasos, tal y como muestra la figura 1.6, para el caso justo mencionado. Este hecho es importante que lo tenga presente, ya que tiene consecuencias muy directas a la hora de implementar un algoritmo cuando se trata de un programa de ordenador.

Figura 1. 6. Flujo de control del algoritmo basado en diseño descendente para freír zanahorias o cebollas



En conclusión, al descomponer un problema, es especialmente acertado intentar hacerlo de forma que se fuerce la aparición de subproblemas repetidos, y así su resolución se puede reaprovechar en varios lugares.

1.1.3 Aplicación correcta del diseño descendente

Un aspecto que debe tener en cuenta al aplicar diseño descendente es que se trata de una estrategia basada en unas directrices generales para atacar problemas complejos, pero no es ningún esquema determinista que le garantice que siempre obtendrá la mejor solución. Esto significa que, partiendo de un mismo problema, diferentes personas pueden llegar a conclusiones distintas sobre cómo llevar a cabo la descomposición. De entre todas las soluciones distintas posibles, algunas pueden considerarse mejores que otras. De hecho, nada impide, a partir ya de una solución concreta, aplicar refinamientos que la mejoren. Por tanto, es interesante poder evaluar si la descomposición que ha hecho va por buen camino o no.

Algunos de los criterios en que puede basarse para realizar esta evaluación son los siguientes:

- Si un problema que parece a priori bastante complejo se descompone en muy pocos niveles, quizás vale la pena echar un segundo vistazo. Inversamente, si un problema no demasiado complejo tiene demasiados niveles, quizás se ha ido demasiado lejos en la descomposición.
- Ver si el número de pasos incluidos en cada uno de los subproblemas no es excesivamente grande y es fácil de seguir y entender. De lo contrario, quizás todavía haría falta aplicar nuevos niveles de descomposición.
- Repasar que los nombres asignados a los subproblemas sean autoexplicativos y expresen claramente la tarea que están resolviendo. Sin siquiera leer sus pasos, habría que entender perfectamente qué se logra al resolverlos. De lo contrario, puede que la descomposición no esté agrupando conjuntos de pasos realmente relacionadas entre sí.
- Si absolutamente ninguno de los subproblemas es reutilizado en ninguna parte, especialmente en descomposiciones en muchos niveles, es muy posible que no se haya elegido correctamente la forma de descomponer algunos subproblemas.
- Vinculado al punto anterior, la aparición de subproblemas muy similares o idénticos, pero tratados por separado en distintos lugares, también suele ser indicio de que no se está aplicando la capacidad de reutilizar subproblemas correctamente.

1.2 Diseño descendente aplicado a la creación de programas

Una vez que dispone de un marco de referencia general sobre cómo aplicar diseño descendente, es el momento de aplicar la misma técnica para la creación de un programa. Para empezar, se muestra la aplicación de diseño descendente sobre un programa de complejidad baja y que ya conoce, de modo que el resultado de la descomposición sea muy simple. Éste sirve como hilo argumental para poder analizar algunos de los aspectos importantes del diseño descendente y ver cómo es

posible implementar la descomposición en subproblemas resultantes, de acuerdo a la sintaxis del lenguaje Java.

El problema de lo que se parte es, o debería ser, un conocido suyo: un programa que, a partir de una lista de 10 valores enteros, los muestre por pantalla ordenados.

Antes de empezar, es una buena idea pensar qué tipo de datos es necesario manipular y cómo se almacenarán, ya que esto le permitirá evaluar en cada paso de la descomposición si un problema es demasiado complejo todavía o no. En este caso es necesario manipular una lista de enteros, por lo que lo más sensato sería almacenarla en forma de un array.

Como se ha visto para el caso general de aplicación de diseño descendente, una buena forma de descomponer el problema en subproblemas es establecer cuáles son las partes diferenciadas, o etapas, que lo componen. Cada etapa se corresponderá a un subproblema a resolver. Lo importante en este paso es que cada subproblema corresponda siempre a una tarea concreta con un objetivo a resolver claramente diferenciado. Debe ser fácil asignarle un nombre. De lo contrario, seguramente no se está haciendo bien la descomposición.

Para este programa, se puede considerar que lo divide en tres partes, o subproblemas, a resolver:

- leer la lista de enteros,
- ordenarla, y
- mostrarla por pantalla.

No todas las
descomposiciones deben
tener siempre muchos niveles.
Si el programa es simple,
habrá pocos.

Una vez llegados a ese primer nivel de descomposición, es el momento de plantearse si cada subproblema es demasiado complejo o no. En este caso, se supone que ya domine el conjunto de instrucciones que se necesitan, cómo leer una lista de enteros de longitud conocida (guardándolos en un array), cómo ordenar un array (usando el algoritmo de la burbuja) y cómo mostrarlo por pantalla (mediante un recorrido). Por tanto, se puede considerar que todos los subproblemas planteados no son excesivamente complejos y el proceso de descomposición termina.

1.2.1 Declaración de métodos

Descompuesto el problema general, es el momento de crear el programa que lo resuelve mediante código fuente. Por eso, habrá que decidir, para cada uno de los subproblemas que se han detectado, qué instrucciones es necesario ejecutar para resolverlo individualmente. Los lenguajes de programación permiten una implementación directa de este proceso, al ofrecer mecanismos para agrupar o catalogar bloques de instrucciones y etiquetarlos con un identificador, de acuerdo a su subproblema asociado.

Por lo general dentro de los lenguajes de programación, se llama una función a un conjunto de instrucciones con un objetivo común que se declaran de manera explícitamente diferenciada dentro del código fuente mediante una etiqueta o identificador.

Por tanto, para cada subproblema a resolver, dentro de su código fuente se deberá definir una función diferente. En el lenguaje Java, estos conjuntos de instrucciones se les llama métodos, en lugar de funciones, pero a efectos prácticos, los puede considerar el mismo. Este término no es nuevo, puesto que ha sido usado con anterioridad bajo dos contextos diferentes, si bien nunca se había entrado en mucho detalle en su descripción ni se había justificado su formato.

En algunas partes de la literatura, a las funciones que cumplen ciertas propiedades se les llama acciones.

- Cuando se habla de método principal, se trata de un conjunto de instrucciones que, etiquetadas bajo un identificador llamado main, resuelven el problema general (o sea, todo el programa). Dado que hasta el momento no se había aplicado diseño descendente, no había subproblemas, y por tanto en su código fuente sólo había definido este único método. No hacía falta otro.
- Cuando se habla de la invocación de un método sobre valores de ciertos tipos de datos complejos, como las cadenas de texto (String), se trata de ejecutar un conjunto de instrucciones con un objetivo común: transformar la cadena de texto u obtener datos contenidos.

Como puede ver, a pesar de no haber entrado en detalle, la forma en que se han usado hasta el momento los métodos es coherente con la definición que apenas se ha presentado. A partir de ahora empezará a estudiarlos con más profundidad.

La declaración básica de un método se realiza usando la sintaxis que se muestra a continuación. Como puede ver, su formato es muy parecido a cómo se declara el método principal (¡pero no exactamente igual, alerta!):

```
1 public void nombreMetodo() { //Aquí dentro irán sus
2     instrucciones //...
3
4 }
```

Esta declaración se puede llevar a cabo en cualquier lugar del archivo de código fuente, siempre que sea entre las claves que identifican el inicio y fin de archivo (public class NomClasse { ... }) y fuera del bloque de instrucciones método principal, o cualquier otro método. Normalmente, suele realizarse inmediatamente a continuación del método principal. La declaración debe seguir exactamente este formato. La única parte que puede modificar es nombreMetode, que no es más que un identificador, como el de una variable, y por tanto puede elegir el que desee. Sin embargo, siempre debería procurar usar algún texto que sea autoexplicativo.

Los identificadores de los métodos siguen las mismas convenciones de código que las variables (lowerCamelCase).

De acuerdo a la descomposición propuesta, en el código fuente del programa de ordenación habrá declarados tres métodos, uno asociado a cada subproblema. Éstos podrían ser:

```

1 public class OrdenarDescendente {
2     public static void main(String[] args) {
3         //Instrucciones del método principal (problema general)
4         //...
5     }
6     //Método que resuelve el subproblema de leer la lista.
7     public void leerLista() {
8         //Instrucciones del método
9         //...
10    }
11    //Método que resuelve el subproblema de ordenar la lista.
12    public void ordenarLista() {
13        //Instrucciones del método
14        //...
15    }
16    //Método que resuelve el subproblema de mostrar la lista por pantalla.
17    public void mostrarLista() {
18        //Instrucciones del método
19        //...
20    }
21 }

```

1.2.2 Cambios en el método principal al declarar otros métodos

Antes de continuar, es necesario presentar un cambio necesario en el formato de los suyos programas cuando se quiere declarar otros métodos, asociados a subproblemas, aparte del método principal.

Para diferenciar los métodos que resuelven subproblemas del método principal, y evitar confusiones, puede referirse a ella como métodos auxiliares.

Concretamente, por las características del lenguaje Java, es necesario que el método principal tenga un formato muy concreto. De lo contrario, habrá un error de compilación en futuros pasos del proceso. Todo el código que iría normalmente dentro el bloque de instrucciones del método principal se ubica en un nuevo método auxiliar, y dentro del método principal simplemente se invoca este nuevo método. De hecho, no es imprescindible que conozca los detalles de los motivos por los que es necesario hacer ese cambio. Simplemente puede usar el siguiente código de ejemplo como plantilla para generar sus programas, teniendo en cuenta que todo el código que pondría normalmente en el método principal, ahora irá al método inicio.

```

1 public class OrdenarDescendente {
2     public static void main (String[] args) {
3         //Aquí hay que usar el nombre de la clase que está creando.
4         OrdenarDescendente programa = new OrdenarDescendente();
5         programa.inicio();
6     }
7     public void inicio() {
8         //Instrucciones del método principal (problema general)
9         //...
10    }
11    //Resto de métodos
12    //...
13 }

```

Al usar este código como plantilla, note que en la línea siguiente habría que poner el nombre de la clase que está editando, en lugar de OrdenarDescendente:

```
1 OrdenarDescendente programa = new OrdenarDescendente();
```

1.2.3 Accesibilidad de variables en una clase

En el momento en que se aplica diseño descendente y las partes del código de un programa se descomponen con métodos, aparece un problema. Los diferentes métodos que defina sirven para procesar una información común a los tres, en este caso, la lista de enteros, en forma de array. Esto significa que necesitará manipular esta variable en los diferentes métodos, de modo que sus valores sean compartidos y accesibles por todos ellos. Ahora bien, antes de empezar, habrá que decidir exactamente donde se declarará.

Para tomar correctamente esta decisión es necesario hacer memoria y recordar el concepto de ámbito de una variable: dada una variable, sólo se considerará declarada desde de la línea donde se ha hecho hasta encontrar la siguiente clave cerrada (}). Si se fija, la consecuencia directa de esto es que, dado que cada método toma la forma de un bloque de instrucciones cerrado entre claves ({ ... }), si una variable se declara dentro de alguno método, sea cual sea, ésta no se consideró declarada en ninguno de los demás.

Hay diferentes formas de solucionar este problema. Por el momento verá la más simple. Cualquier dato que deba ser accedido en más de un subproblema para resolverlo, será necesario declararla como una variable global.

Una variable global es una variable que puede ser accedida desde cualquiera instrucción dentro de un mismo archivo de código fuente. Su ámbito es todo el archivo.

En contraposición a las variables globales, están las variables locales, que son las que ha usado hasta ahora: variables con un ámbito local en un bloque concreto de código.

En Java, la sintaxis para declarar una variable global es muy similar a la que se ha visto hasta ahora, sólo varía el hecho de que es necesario añadir la palabra clave `private` antes de la declaración y lugar donde declararla. Este último punto es, de hecho, lo más importante si desea que una variable se considere global.

```
1 private tipo nombreVariable = valorInicial;
```

En este caso, la declaración debe realizarse fuera de todos los métodos, pero dentro del bloque de claves que delimita la clase, exactamente igual que cuando declaráis constantes. Por ejemplo, si se quiere declarar un array de enteros llamado `listaEnters`, se podría hacer:

```
1 public class OrdenarDescendente {  
2     //Variable global  
3     private int[] listaEnters = new int[10];  
4     public static void main (String[] args) {  
5         OrdenarDescendente programa = new OrdenarDescendente();  
6         programa.inicio();  
7     }  
8     public void inicio() {  
9         //Instrucciones del método principal (problema general)  
10        //...  
11    }  
12    //Método con las instrucciones para leer la lista.  
13    public void leerLista() {  
14    }  
15    //Método con las instrucciones para ordenar la lista.  
16    public void ordenarLista() {
```

```
17     }  
18     //Método con las instrucciones para mostrar la lista por pantalla. public void mostrarLista()  
19     { }  
20  
21 }
```

Al ser global, la variable `listaEnters` será accesible desde cualquier instrucción dentro del código. Merece la pena remarcar que Java no es imprescindible declararla al inicio del código fuente para poder ser usada libremente. Por ejemplo, si se declarara en la última línea, justo después de la declaración del método `mostrarLista`, se seguiría considerando declarada para todo el archivo. Esto es una ligera diferencia respecto a las variables locales, que sólo tienen vigencia a partir de la línea de código en la que se han declarado. De todas formas, por convención, suelen declararse al principio de todo, de modo que el código queda ordenado: primero variables globales y constantes, y después métodos.

Antes de continuar, es importante remarcar que el uso de variables globales sólo se considera pulido en casos como éste, donde existe un dato que debe ser manipulado en diferentes subproblemas. Para otros datos de uso limitado a un único método (contadores o semáforos de bucles, resultados temporales de operaciones, etc.), será necesario declararlos en el bloque de código correspondiente y nunca como una variable global.

1.2.4 Codificación de métodos

Una vez que ya se dispone del método principal adaptado, los datos generales del programa declarados como variables globales y la declaración de un método asociado a cada subproblema resultando la descomposición del problema general, ya puede proceder a escribir las instrucciones de cada método. En este aspecto, las claves que delimitan un método (`public void nombreMetodo() { ... }`) conforman el bloque de instrucciones que resuelve ese problema concreto.

El orden en el que será necesario resolverlo es por niveles de descomposición, normalmente comenzando por el nivel más bajo, ya que es el más intuitivo. Por las características de la descomposición mediante diseño descendente, si la ha aplicado correctamente, la resolución de cada método debería ser una tarea totalmente autocontenida e independiente. Por tanto, debería poder resolver en cualquier orden los métodos asociados a problemas de un mismo nivel de descomposición. Si aparece alguna dependencia es que la descomposición no es correcta.

En el caso del ejemplo, sólo existen dos niveles, el problema general y el primer nivel de descomposición. En el nivel más bajo, el método `leerLista` tendrá las instrucciones que leen 10 enteros desde el teclado y los guardan en el array, el método `ordenarLista`, las que ordenan el array y el método `mostrarLista`, las que lo muestran por pantalla. Por tanto, el código podría ser:

```
1 importe java.util.Scanner; 2 public class  
OrdenarDescendente {  
3     int[] listaEnters = new int[10]; public static  
4     void main (String[] args) { OrdenarDescendente programa  
5         = new OrdenarDescendente(); programa.inicio();  
6 }
```

```

7     }
8     public void inicio() {
9         //Instrucciones del método principal (problema general)
10        //...
11    }
12    //Método con las instrucciones para leer la lista.
13    public void leerLista() {
14        System.out.println("Escribe 10 valores enteros y pulsa retorno.");
15        Scanner lector = new Scanner(System.in);
16        int índice = 0;
17        while (index < listaEnters.length) {
18            if (lector.hasNextInt()) {
19                listaEnters[index] = lector.nextInt();
20                index++;
21            } else {
22                lector.next();
23            }
24        }
25        lector.nextLine();
26    }
27    //Método con las instrucciones para ordenar la lista.
28    public void ordenarLista() {
29        for (int i = 0; i < listaEnters.length - 1; i++) {
30            for (int j = i + 1; j < listaEnters.length; j++) {
31                //La posición tratada tiene un valor más alto que el de la búsqueda... Los
                //intercambiamos.
32                if (listaEnters[i] > listaEnters[j]) {
33                    //Para intercambiar valores es necesaria una variable auxiliar
34                    int cambio = listaEnters[i];
35                    listaEnters[i] = listaEnters[j];
36                    listaEnters[j] = cambio;
37                }
38            }
39        }
40    }
41    //Método con las instrucciones para mostrar la lista por pantalla.
42    public void mostrarLista() {
43        System.out.print("El array ordenado es: [ ");
44        for (int i = 0; i < listaEnters.length; i++) {
45            System.out.print(listaEnters[i] +
46                             " ");
47        }
48        System.out.println("]");
49    }

```

1.2.5 Invocación de métodos

Una vez resueltos todos los métodos de un nivel dado, se puede proceder a resolver los del nivel superior. Ahora bien, al hacerlo, recuerde que tiene la posibilidad de usar la solución de cualquier subproblema de nivel inferior. Por ejemplo, éste era el caso de aprovechar saber preparar el aceite en la sartén para solucionar cómo freír las zanahorias en la receta de cocina.

Dentro del código fuente, esto se hace invocando alguno de los métodos que ha codificado. Por hacerlo, basta con poner una instrucción que contiene el nombre del método a invocar, dos paréntesis y el punto y coma de final de sentencia. O sea:

```
1 nombreMetodo();
```

A efectos prácticos, cada vez que se invoca un método, el programa ejecuta las instrucciones que hay codificadas dentro de ese método, desde la primera hasta

la última. Cuando acaba de ejecutar la última instrucción del método, entonces el programa procede a ejecutar la línea inmediatamente posterior a la invocación al método.

Si vuelve al ejemplo, ahora ya sólo le quedaría completar el código asociado al problema general (método inicio). Para ello, es posible invocar a leerLista, ordenarLista y mostrarLista. Dado este hecho, el programa final ya sería el siguiente. Compílelo y ejecútelo para ver que es así.

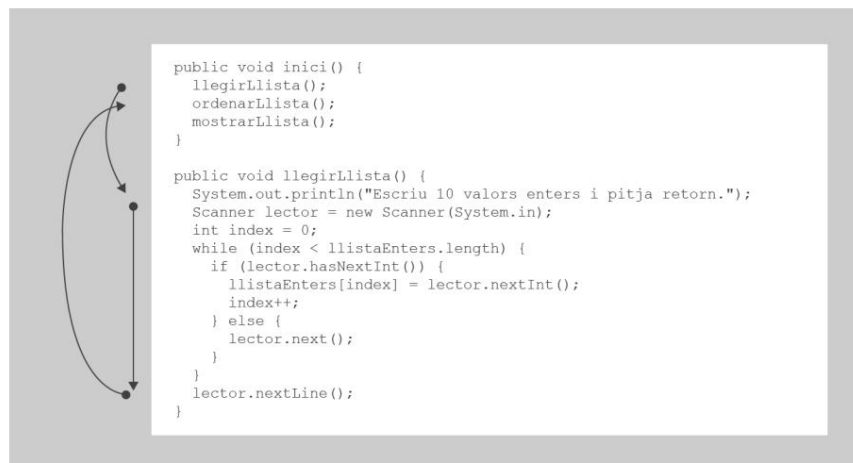
```
1 importe java.util.Scanner;
2 public class OrdenarDescendente {
3     int[] listaEnters = new int[10];
4     public static void main (String[] args) {
5         OrdenarDescendente programa = new OrdenarDescendente();
6         programa.inicio();
7     }
8     public void inicio() {
9         leerLista();
10        ordenarLista();
11        mostrarLista();
12    }
13    //Método con las instrucciones para leer la lista.
14    public void leerLista() {
15        System.out.println("Escribe 10 valores enteros y pulsa retorno.");
16        Scanner lector = new Scanner(System.in);
17        int índice = 0;
18        while (index < listaEnters.length) {
19            if (lector.hasNextInt()) {
20                listaEnters[index] = lector.nextInt();
21                index++;
22            } else {
23                lector.next();
24            }
25        }
26        lector.nextLine();
27    }
28    //Método con las instrucciones para ordenar la lista.
29    public void ordenarLista() {
30        for (int i = 0; i < listaEnters.length - 1; i++) {
31            for(int j = i + 1; j < listaEnters.length; j++) {
32                //La posición tratada tiene un valor más alto que el de la búsqueda... Los
                //intercambiamos.
33                if (listaEnters[i] > listaEnters[j]) {
34                    //Para intercambiar valores es necesaria una variable auxiliar
35                    int cambio = listaEnters[i];
36                    listaEnters[i] = listaEnters[j];
37                    listaEnters[j] = cambio;
38                }
39            }
40        }
41    }
42    //Método con las instrucciones para mostrar la lista por pantalla.
43    public void mostrarLista() {
44        System.out.print("El array ordenado es: [ ");
45        for (int i = 0; i < listaEnters.length; i++) {
46            System.out.print(listaEnters[i] + " ");
47        }
48        System.out.println("");
49    }
50 }
```

Una vez que ya dispone de todo el marco de referencia sobre cómo queda distribuido un programa generado por la descomposición del problema usando diseño descendente, es interesante ver también cuál es el flujo de control de las instrucciones cuando éste ejecuta la invocación a un método. Esto se muestra en la figura 1.7 para el caso de la invocación al método leerLista.

Reto 1: Modifique el programa de ejemplo de forma que haga lo siguiente.

Después de mostrar la lista ordenada, en una nueva línea, debe decir cuántos de los valores son inferiores a la mitad del mayor valor almacenado. Aplique diseño descendente para añadir esta nueva tarea, declarando e invocando los nuevos métodos que haga falta.

Figura 1. 7. Flujo de control de las instrucciones cuando se invoca el método leerLista.



1.2.6 Inicialización diferida de variables

La necesidad de declarar variables globales comporta una problemática en casos muy específicos. Hasta ahora, en el momento de declarar una variable, inmediato le ha asignado un valor inicial. Este valor podía ser tanto el resultado de asignar directamente un literal como una expresión o una entrada de datos por parte del usuario. Sin embargo, al declarar una variable como global, sólo puede ser inicializada directamente mediante un literal o expresiones donde se usan otras variables globales. No existe la posibilidad de hacerlo mediante un valor que dependa de una entrada, por teclado o de los argumentos del método principal. Esto significa que la declaración de la variable y la asignación del valor que se quiere tratar realmente no puede realizarse en la misma instrucción. Sin embargo, siempre que se declara una variable es necesario asignarle un valor inicial.

Para el caso de variables de tipos primitivos, resolver este problema es simple. Por convención, se le asigna inicialmente el valor 0 y más adelante ya se sobrescribirá su valor con otro válido. Ahora bien, para el caso de variables complejas (como las de tipo array o String), es necesario asignar un valor especial que en Java se llama null. Esta cadena de texto es una palabra reservada del lenguaje que sirve para decir que, de momento, la variable está declarada pero más adelante ya se le asignará un valor correcto, tan pronto como sea posible.

Operar con cualquier variable tipo complejo con un valor null asignado siempre resultará en un programa erróneo.

Por ejemplo, suponga que desea modificar el programa anterior de forma que, en lugar de entrar diez valores por el teclado, puede entrar un número arbitrario. En

partir de la secuencia escrita, el primer valor indicará cuántos enteros hay que leer todo seguido por el teclado. En un caso como éste, es imposible inicializar el array con un tamaño concreto, ya que ésta depende de una entrada por el teclado. Pero el programa requiere que éste sea declarado como una variable global. Por tanto, hay que diferir la inicialización.

El código que resuelve esta situación sería el siguiente. Dado que leer la secuencia de enteros del teclado ahora es un problema más complejo, es necesario leer el tamaño y los valores. Se ha aplicado diseño descendente para dividirlo en dos subproblemas: leer el tamaño de la secuencia y la secuencia propiamente. Compruebe que funciona en su entorno de trabajo.

```
1 //Variable global. Array no inicializado.
2 private int[] listaEnters = null;
3 //Al aplicar diseño descendente, ahora es necesario declarar "lector" como global
4 Scanner lector = new Scanner(System.in);
5 public static void main (String[] args) {
6     OrdenarDescendentVariable programa = new OrdenarDescendentVariable();
7     programa.inicio();
8 }
9 public void inicio() {
10     leerLista();
11     ordenarLista();
12     mostrarLista();
13 }
14 //Método con las instrucciones para leer la lista.
15 //El primer valor será la longitud
16 public void leerLista() {
17     System.out.println("Escribe una lista de valores enteros y pulsa retorno.");
18     System.out.println("El primer valor indica el tamaño de la secuencia.");
19     leerMida();
20     leerValores();
21 }
22 public void leerMida() { //Lectura //Metodo que lee el primer valor
23
24     int tamaño = 0;
25     if (lector.hasNextInt()) {
26         tamaño = lector.nextInt();
27     } else {
28         lector.next();
29     }
30     listaEnters = new int[tamaño]; //Inicialización diferida del array
31 }
32 public void leerValores() {
33     int índice = 0;
34     while (index < listaEnters.length) {
35         if (lector.hasNextInt()) {
36             listaEnters[index] = lector.nextInt();
37             index++;
38         } else {
39             lector.next();
40         }
41     }
42     lector.nextLine();
43 } //El resto de métodos no cambian ...
44 }
```

1.3 Un ejemplo más complejo

Una vez visto un ejemplo sencillo de aplicación de diseño descendente, básicamente con el objetivo de introducir la sintaxis para la declaración de métodos y variables globales en Java, es el momento de proponer otro más complejo, que requiera varios grados de descomposición.

Lo que se quiere hacer es un gestor de registro de temperaturas tomadas semanalmente por un observatorio. Se presupone que el programa se pone en marcha a principios de año (1 de enero) y al principio de cada semana. A lo largo de 52 semanas que tiene un año, se van registrando las temperaturas medidas cada día de la semana anterior (o sea, siete en total cada vez). Cada vez que se realiza un registro, sabiendo que ha pasado una semana, el programa calcula automáticamente qué día y mes es el actual. A partir de estos datos, es posible consultar en cualquier momento cuál ha sido la temperatura media y la diferencia entre el valor máximo y mínimo registrados.

Al hacerlo, la fecha actual también se muestra en pantalla.

Todas estas acciones se llevan a cabo usando un menú. Evidentemente, la aplicación debe ser lo suficientemente robusta como para tratar casos erróneos (por ejemplo, consultar valores cuando todavía no hay fecha registrada, o intentar registrar como temperatura valores de tipo incorrecto).

Para dejar más claro el comportamiento esperado, a continuación se muestra un prototipo de lo que se esperaría mostrar con su ejecución:

```
1 Bienvenido al registro de temperaturas
2 -----
3 [RT] Registrar temperaturas semanales.
4 [MJ] Consultar temperatura media.
5 [DF] Consultar diferencia máxima.
6 [FI] Salir.
7 Opción: MJ 8
No hay temperaturas registradas.
9
10 Bienvenido al registro de temperaturas
11 -----
12 [RT] Registrar temperaturas semanales.
13 [MJ] Consultar temperatura media.
14 [DF] Consultar diferencia máxima.
15 [FI] Salir.
16 Opción: RT 17
Escribe las temperaturas de esta semana: 18 20,5 21,1 21 21,7
20,9 20,6 19,9
19
20 Bienvenido al registro de temperaturas
21 -----
22 [RT] Registrar temperaturas semanales.
23 [MJ] Consultar temperatura media.
24 [DF] Consultar diferencia máxima.
25 [FI] Salir.
26 Opción: MJ 27
Hasta la fecha 8 de enero la media ha sido de 20.814285 grados.
28
29 Bienvenido al registro de temperaturas
30 -----
31 [RT] Registrar temperaturas semanales.
32 [MJ] Consultar temperatura media.
33 [DF] Consultar diferencia máxima.
```

```
34 [FI] Salir.  
35 Opción: DF 36  
Hasta la fecha 8 de enero la diferencia máxima ha sido de 1.8000011 grados.  
37  
38 Bienvenido al registro de temperaturas  
39 -----  
40 [RT] Registrar temperaturas semanales.  
41 [MJ] Consultar temperatura media.  
42 [DF] Consultar diferencia máxima.  
43 [FI] Salir.  
44 Opción: FI
```

1.3.1 Descomposición de problema

Una vez planteado con cierto detalle el problema a resolver (qué debe hacer el programa), es posible iniciar la descomposición mediante diseño descendente. Para ver con más detalle el proceso, esta vez se irá haciendo despacio y nivel por nivel.

1. Identificación de los datos a tratar

Antes de empezar la descomposición, como paso previo, es interesante establecer qué tipo de datos es necesario manipular y cómo almacenarlos dentro del programa. De esta forma, resulta más fácil evaluar por cada subproblema qué debe llevar a cabo y si se trata de una tarea compleja o no. En este caso, es necesario gestionar de una lista de temperaturas, que se puede almacenar usando un array de reales, y una fecha dentro de un mismo año, que se puede almacenar usando dos enteros, día y mes. El array deberá tener espacio para almacenar los valores de los días a 52 semanas ($52 \times 7 = 364$) y habrá que controlar el hecho de que existen posiciones “vacías” y otras con valores correctos asignados. Por ejemplo, después de la primera semana, sólo los 7 primeros valores, las posiciones 0 a 6, son válidos. El resto de posiciones carecen de valores válidos asignados.

Cuando llegue el momento habrá que considerar si declararlos como variables globales, dependiendo de si estos datos se usan dentro de más de un subproblema o no.

2. Primer nivel de descomposición

El resultado de aplicar el primer nivel de descomposición puede resultar en muchos o pocos subproblemas dependiendo del grado de granularidad con el que decida tratar las tareas que realiza el programa. Inicialmente, es recomendable no usar una granularidad alta y mantener un alto nivel de abstracción. Normalmente, es importante no bajar rápidamente de nivel y empezar a resolver problemas muy concretos en un solo pase. Una estrategia para evitar esto es plantearse qué acciones es necesario emprender antes de poder llevar a cabo otras.

Partiendo de la descripción del problema, una posible descomposición en niveles sería la siguiente, todavía bastante general. El programa es básicamente una estructura de repetición que va iterando sobre estas dos tareas:

- Mostrar menú.
- Tratar orden.

Estas iteraciones se irán repitiendo hasta cumplir la condición que desea finalizar el programa. De entrada, se puede decidir que esto se llevará a cabo con una variable de control de tipo semáforo.

Hasta cierto punto, para un diseñador novel, sería comprensible proponer ya en el primer nivel resolver subproblemas tales como el cálculo de las temperaturas mínimas y máximas, puesto que son aspectos que resaltan claramente en el enunciado.

Pero si reflexiona, se dará cuenta de que para poder llevar a cabo estas tareas hay condiciones previas que antes hay que cumplir: que el usuario haya seleccionado una opción. Por tanto, esto significa que gestionar el menú y ejecutar las opciones tiene relación de jerarquía dentro del diseño: primero lea la opción y luego la ejecute. Por tanto, no se encuentran en el mismo nivel.

3. Segundo nivel de descomposición

Para ver si es necesario un segundo nivel es necesario evaluar si los subproblemas propuestos en el primer nivel son demasiado complejos todavía. Evidentemente, dependiendo de la destreza del programador, lo que se considera complejo puede ser muy relativo. En cualquier caso, y esto es independiente de la habilidad del programador, lo que hay que identificar son tareas claramente diferenciadas que es necesario resolver para solucionar cada subproblema.

- Mostrar menú. Para ello, básicamente basta con imprimir un conjunto de texto en pantalla y ya está. Es una tarea muy simple que puede llevarse a cabo mediante sucesivas invocaciones en `System.out.println`. Por tanto, no hace falta descomponerla más.
- Tratar orden. Hay que leer el orden por el teclado y analizar si lo que se lee se corresponde a alguna de las cuatro órdenes posibles. Esto puede hacerse con una estructura de selección. Entonces, según lo leído, hay que realizar tareas totalmente diferentes. Claramente, se trata de una tarea compleja que es necesario descomponer. La forma más lógica de hacerlo, inicialmente, podría ser por cada tarea a realizar.

A partir de este análisis, la descomposición hasta el segundo nivel quedaría como:

1. Mostrar menú.
 - (a) Entrar registro de temperaturas semanales.
 - (b) Mostrar temperatura media.
 - (c) Mostrar diferencia máxima.
 - (d) Finalizar ejecución.
2. Tratar orden.

4. Tercer nivel de descomposición

De nuevo, se realiza una iteración sobre los subproblemas de segundo nivel para ver si presentan tareas complejas o no. En función de esto, habrá que decidir si es necesario seguir descomponiéndolos.

- Entrar registro de temperaturas semanales. Para ello es necesario resolver dos tareas. Por un lado, leer las temperaturas y ponerlas en el array de temperaturas. Además, también es necesario ir actualizando la fecha actual cada vez que se leen datos (avanzarla 7 días). Esto no es simple, ya que es necesario controlar el caso de qué día termina cada mes (28, 30 o 31 días).
- Mostrar temperatura media. Este problema puede descomponerse en dos. Por un lado, se pide mostrar la fecha de forma que el mes se muestre en formato texto, partiendo de un número. Por otro lado, es necesario mostrar el cálculo que se pide (sumar todos los valores al array de temperaturas y dividirlos por su número).
- Mostrar diferencia máxima. Este caso es exactamente igual que el anterior, sólo que el cálculo es diferente (buscar con un único recorrido los valores máximo y mínimo y calcular su diferencia).
- Finalizar ejecución. Básicamente, sería cambiar el valor de la variable de control de tipo semáforo que controla la estructura de repetición donde se englobarán "Mostrar menú" y "Tratar orden". Es muy simple.

Según este análisis, la descomposición hasta el tercer nivel quedaría así:

1. Mostrar menú.
2. Tratar orden.
 - (a) Entrar registro de temperaturas semanales.
 - i. Leer temperaturas del teclado.
 - ii. Actualizar fecha actual.
 - (b) Mostrar temperatura media.
 - i. Mostrar fecha actual.
 - ii. Calcular temperatura media.
 - (c) Mostrar diferencia máxima.
 - i. Mostrar fecha actual.
 - ii. Calcular diferencia máxima.
 - (d) Finalizar ejecución.

5. Cuarto nivel de descomposición

De nuevo, corresponde estudiar si es necesario realizar un nuevo nivel de descomposición según el grado de complejidad de los subproblemas de tercer nivel. Un punto interesante que ahora se encuentra es el hecho de que se pueden localizar subproblemas repetidos. Mostrar la fecha actual es una tarea a realizar en diferentes lugares. Por tanto, sólo habrá que resolver este subproblema una única vez.

- Leer temperaturas del teclado. Aunque no se hace en pocas líneas de código, sabe leer 7 valores de tipo real y asignarlos a un array. Por tanto, no es una tarea especialmente compleja que valga la pena descomponer más.
- Actualizar fecha actual. Se trata de incrementar el día y, dependiendo del mes, con una estructura de selección, ver si se ha adelantado a un nuevo mes. No se compone de pasos muy complejos.
- Mostrar fecha actual. Se trata de mostrar el día directamente y mostrar cierto texto según el valor numérico del más. Esto podría hacerse con una estructura de selección. Por tanto, tampoco se compone de pasos muy complejos.
- Calcular temperatura media. Es un cálculo sobre los valores registrados, haciendo un recorrido sobre el array. Es simple.
- Calcular diferencia máxima. Exactamente un caso muy parecido al anterior oro.

Dado que todos los subproblemas del tercer nivel ya son simples y resuelven una tarea muy concreta y autocontenida, no es necesario un cuarto nivel de descomposición. Hais terminado.

1.3.2 Esqueleto de la clase

Una vez identificados los datos que se quieren tratar y finalizado el proceso de descomposición inicial, es posible crear un esqueleto de la clase, sólo con la declaración de variables globales y métodos necesarios. Cada subproblema equivale a un método a declarar. Si el esqueleto está correctamente declarado, debería ser posible compilar el código fuente, aunque al ejecutarse no haría absolutamente nada todavía. Se trata sólo de una organización general del código fuente.

En este ejemplo el esqueleto quedaría así:

```
1 public class RegistroTemperaturas {  
2  
3     //Constantes  
4     private static final int MAX_SETMANES = 52;  
5  
6     //Variables globales  
7     private int numTemperaturas = 0;  
8     private float[] temperaturas = new float[MAX_SETMANES * 7];  
9     private int dia = 1;  
10    private int mes = 1;  
11  
12    //Métodos asociados al problema general  
13    public static void main (String[] args) {  
14        RegistroTemperaturas programa = new RegistroTemperaturas();  
15        programa.inicio();  
16    }  
17    public void inicio() {  
18    }  
19  
20    //Métodos asociados al primer nivel de descomposición  
21    public void mostrarMenu() {  
22    }
```

```
23 public void tratarOpcio() {  
24 }  
25  
26 //Métodos asociados al segundo nivel de descomposición  
27 public void registroTemperaturasSemanales() {  
28 }  
29 public void mostrarMedia() {  
30 }  
31 public void mostrarDiferencia() {  
32 }  
33 public void finalizarEjecución() {  
34 }  
35  
36 //Métodos asociados al tercer nivel de descomposición  
37 //etc.  
38 }
```

Reto 2: Complete el código fuente del esqueleto del ejemplo con la declaración de los métodos en el tercer nivel de descomposición. Sus nombres serán: leerTemperaturasTeclado, incrementarData, mostrarData, calculaMedia y calculaDiferencia.

1.3.3 Implementación del tercer nivel de descomposición

Para codificar la descomposición de este problema, también se empezará desde los métodos asociados a los subproblemas del nivel más bajo de descomposición y se irá subiendo poco a poco hasta llegar a la resolución del problema general, que corresponde en el método principal. Por tanto, hay que codificar los métodos:

- leerTemperaturasTeclado: registra 7 temperaturas, o sea, lee 7 valores reales y los guarda en el array de temperaturas.
- incrementarData: dada una fecha, suma 7 a su valor.
- mostrarData: muestra la fecha actual, en formato: Número del día de Nombre del mes.
- calculaMedia: muestra por pantalla la media aritmética de temperatura-nada del registro.
- calculaDiferencia: muestra por pantalla la diferencia de temperaturas entre los valores máximo y mínimo del registro.

Una propuesta de cómo hacer su código sería la siguiente: dado que los métodos son blogs autocontenidos de código, una vez incorporados al código fuente del programa, es posible compilarlo perfectamente sin problemas, aunque, evidentemente, el programa todavía no hará nada si se ejecuta. Añádelos y compruebe que es así.

```
1 //Métodos asociados al tercer nivel de descomposición  
2 public void leerTemperaturasTeclado() {  
3     System.out.println("Escribe las temperaturas de esta semana:");  
4     Scanner lector = new Scanner(System.in);  
5     int numLeídas = 0;  
6     while (numLeídas < 7) {  
7         if (lector.hasNextFloat()) {
```

```
8         temperaturas[numTemperaturas] = lector.nextFloat();
9         numLeídas++;
10        numTemperaturas++;
11    } else {
12        lector.next();
13    }
14    }
15    lector.nextLine();
16    }
17    public void incrementarData() {
18        //¿Cuántos días tiene este mes?
19        int díasEsteMes = 0;
20        if (mes == 2) {
21            díasEsteMes = 28;
22        } else if ((mes == 4) || (mes == 6) || (mes == 9) || (mes == 11)) {
23            díasEsteMes = 30;
24        } else {
25            díasEsteMes = 31;
26        }
27        día = día + 7;
28        //¿Hemos pasado de mes?
29        if (día > díasEsteMes) {
30            día = día - díasEsteMes;
31            mas++;
32            //¿Hemos pasado de año?
33            if (mes > 12) {
34                mas = 1;
35            }
36        }
37    }
38    public void mostrarData() {
39        System.out.print(día + " de ");
40        switch(mes) {
41            case 1:
42                System.out.print("Enero"); break;
43            case 2:
44                System.out.print("Febrero"); break;
45            case 3:
46                System.out.print("Marzo"); break;
47            case 4:
48                System.out.print("Abril"); break;
49            case 5:
50                System.out.print("Mayo"); break;
51            case 6:
52                System.out.print("Junio"); break;
53            case 7:
54                System.out.print("Julio"); break;
55            case 8:
56                System.out.print("Agosto"); break;
57            case 9:
58                System.out.print("Septiembre"); break;
59            case 10:
60                System.out.print("Octubre"); break;
61            case 11:
62                System.out.print("Noviembre"); break;
63            case 12:
64                System.out.print("Diciembre");
65        }
66    }
67    public void calculaMedia() {
68        float acumulador = 0;
69        for(int i = 0; i < numTemperaturas; i++) {
70            acumulador = acumulador + temperaturas[i];
71        }
72        System.out.print((acumulador / numTemperaturas));
73    }
74    public void calculaDiferencia() {
75        //Ver Reto 3, más adelante
76        //...
77    }
```

Reto 3: Codifique el método `calculaDiferencia`. Explicado con mayor detalle, este método busca los valores más alto y más bajo de entre los registrados y calcula la diferencia entre ellos. Una vez calculado, muestra el valor resultante por pantalla, tal y como hace `calculaMedia`.

1.3.4 Implementación del segundo nivel de descomposición

Una vez terminada la codificación de los métodos de nivel más bajo de descomposición, se resolverá el nivel inmediatamente superior, paso a paso, sin saltarse ningún nivel. Un aspecto interesante al ir resolviendo niveles superiores es que si la descomposición ha sido apropiada, la dificultad de codificar todos los subproblemas, independientemente del nivel, debería ser similar. En el caso de los niveles más bajos, esto se debe a que se trata de los problemas que ha considerado más simples, como ya ha visto. Sin embargo, en niveles superiores su complejidad también será más baja ya que se dispone de una parte del problema resuelta. Por tanto, la codificación de este segundo nivel no debería resultar tener que hacer métodos mucho más complicados o necesariamente con más código que el paso anterior. De hecho, pueden incluso ser más sencillos.

Los métodos incluidos en este nivel son los asociados a las cuatro opciones posibles dentro del programa:

- `registroTemperaturasSemanales`: gestiona el proceso de registrar temperaturas semanales: leer datos, almacenarlos e incrementar la fecha actual.
- `mostrarMedia`: muestra por pantalla el mensaje de la media de las temperaturas: fecha actual y valor.
- `mostrarDiferencia`: muestra por pantalla el mensaje de la diferencia máxima de las temperaturas: fecha actual y valor.
- `finalizarEjecución`: gestiona la finalización del programa.

La codificación estrictamente de niveles inferiores a superiores de forma totalmente compartimentalizada no siempre es posible. :: A medida que va subiendo de niveles puede encontrar algunos casos especiales, los cuales no son infrecuentes durante un proceso de diseño descendente, y sobre él tendrá que reflexionar.

Por un lado, puede darse el caso en el que habrá que aprovechar un mismo método de niveles inferiores para codificar más de un método del nivel presente. Éste es el caso de `mostrarMedia` y `mostrarDiferencia`, ya que ambos deben mostrar la fecha actual, y por tanto harán uso de `mostrarData`.

Por otra parte, la codificación de un método puede no ser clara a priori, y no será hasta solucionar niveles de descomposición superiores. Éste es el caso de la codificación del método `finalizarEjecución`, ya que controla la finalización de la ejecución del programa principal, un método de nivel superior que todavía no

toca implementar. Éste es un caso en el que la resolución ordenada de nivel inferior a superior individualmente no es perfecta, ya que es necesaria una visión más general del problema. Cuando esto ocurre, hay que dejar el código vacío hasta que la solución quede más clara más adelante.

El código de los cuatro métodos, que prevé las circunstancias apenas descritas, sería lo siguiente. Añádalo al programa y compruebe que compila correctamente.

```
1 //Métodos asociados al segundo nivel de descomposición
2 public void registroTemperaturasSemanales() {
3     //Hay que controlar si habrá espacio para estos 7 registros
4     if ((numTemperaturas + 7) >= temperaturas.length) {
5         System.out.println("No queda espacio para más temperaturas.");
6     } else {
7         leerTemperaturasTeclado();
8         incrementarData();
9     }
10 }
11 public void mostrarMedia() {
12     if (numTemperaturas > 0) {
13         System.out.print("\nHasta la fecha ");
14         mostrarData();
15         System.out.print(" la media ha sido de ");
16         calculaMedia();
17         System.out.println(" grados.");
18     } else {
19         System.out.println("\nNo hay temperaturas registradas.");
20     }
21 }
22 public void mostrarDiferencia() {
23     //Ver Reto 4, a continuación.
24     //...
25 }
26 public void finalizarEjecución() {
27     //Ya se va a pensar en resolver el nivel superior...
28 }
```

Reto 4: Codifique el método mostrarDiferencia.

1.3.5 Implementación del primer nivel de descomposición

En este problema, el primer nivel de descomposición se corresponde con los subproblemas más generales, que engloban todas las funciones del programa. Sólo hace falta implementar dos métodos:

- mostrarMenu: muestra el menú principal por pantalla (sólo imprime cosas por pantalla).
- tratarOpcio: lee el orden y ejecuta el método de segundo nivel correspondiente.

El código que realiza estas tareas es el siguiente. Fíjese que todavía no ha resuelto el método finalizarEjecución, pero dado que se ha declarado el esqueleto, puede invocarse correctamente sin que haya ningún error de compilación. Simplemente, hoy por hoy su invocación es equivalente a no hacer nada (no se ejecuta ninguna instrucción).

ecualsIgnoreCase
Este método de la clase
String compara dos cadenas
de texto ignorando diferencias entre
mayúsculas y minúsculas.

```
1 //Métodos asociados al primer nivel de descomposición
2 public void mostrarMenu() {
3     System.out.println("\nBienvenido al registro de temperaturas");
4     System.out.println("-----");
5     System.out.println("[RT] Registrar temperaturas semanales.");
6     System.out.println("[MJ] Consultar temperatura media.");
7     System.out.println("[DF] Consultar diferencia máxima.");
8     System.out.println("[FI] Salir.");
9     System.out.print("Opción: ");
10 }
11 public void tratarOpcio() {
12     Scanner lector = new Scanner(System.in);
13     String opcio = lector.nextLine();
14     if (opcio.equalsIgnoreCase("RT")) {
15         registroTemperaturasSemanales();
16     } else if (opcio.equalsIgnoreCase("MJ")) {
17         mostrarMedia();
18     } else if (opcio.equalsIgnoreCase("DF")) {
19         mostrarDiferencia();
20     } else if (opcio.equalsIgnoreCase("FI")) {
21         finalizarEjecución();
22     } else {
23         System.out.println("Opción incorrecta\n");
24     }
25 }
```

1.3.6 Implementación del problema general

Por último, ha llegado el momento de implementar el problema general, el método inicio. Dado que todos los subproblemas inferiores ya han sido resueltos, normalmente esta tarea será relativamente simple. Por el funcionamiento que se ha definido por en este programa, lo que debe hacer básicamente es mostrar el menú y leer una orden de forma indefinida, hasta pedir que finalice el programa, o sea una estructura de repetición controlada por una variable de control con funciones de semáforo, que cambiará de estado cuando sea necesario finalizar las iteraciones.

Su código sería el siguiente, donde fin sería la variable de control. Como puede ver, éste es el método más sencillo de todos!

```
1 public void inicio() {
2     while (!fin) {
3         mostrarMenu();
4         tratarOpcio();
5     }
6 }
```

Una vez llegados a este punto, toca dar marcha atrás y recordar que había una tarea pendiente: codificar finalizarEjecución. Sus instrucciones no se podían deducir en su momento, ya que dependían de cómo se resolvería el código del problema general. Ahora que ya sabéis que la finalización del programa depende de una variable de control, ya se puede saber que es necesario cambiar su valor de forma adecuada.

Por tanto, su código será:

```
1 public void finalizarEjecución() { fi = true;
2
3 }
```

Una vez codificado, sólo queda algo, y es declarar esta variable. Dado que se trata de un valor a acceder desde dos métodos diferentes, habrá que hacerlo como una variable global.

```
1 //Variables globales private boolean fi =
2     false; private int numTemperaturas = 0; private float[]
3     temperaturas = new float[MAX_SETMANES * 7]; private int dia = 1;
4     private int mes = 1;
5
6
7     ...
```

1.3.7 Mejoras sobre la solución final

Finalizado el programa, es el momento de ver si funciona. Evidentemente, puede que el código de algún método no haya sido codificado correctamente y sea necesario corregirlo. Para el caso de los programas complejos, el depurador es una herramienta de gran ayuda en esta tarea, complementada con que, usando métodos, es muy fácil identificar la utilidad de cada blog de código.

Aunque el programa funcione, una vez ya dispone de todo el código del programa (ya se han deducido todos los métodos, su código, y las variables globales a usar), vale la pena echar un vistazo general para refinar el resultado. Este proceso de refinamiento se basa en dos principios: eliminar métodos demasiado cortos o simplificar los que son demasiado largos o complejos.

Antes de proceder a mejorar el código, sin embargo, debe tener siempre presente la máxima siguiente: primero es necesario que el programa funcione. Luego ya pensará cómo simplificar el código.

Eliminación de métodos

Una vez codificados todos los métodos, puede que haya alguno que tiene muy pocas líneas -estamos hablando de una o dos. Nunca se reaprovecha, y sólo se usa en un único sitio. Normalmente, cuando esto ocurre se debe a que se ha hilado demasiado delgado en el proceso de descomposición y se ha considerado como subproblema una tarea que es muy sencilla y no tiene suficiente entidad en sí misma. Que esto suceda no significa que el proceso haya sido totalmente incorrecto, puesto que muchas veces, a priori, es imposible saber que esto va a suceder. Muchas veces, sólo una vez codificados todos los métodos puede darse cuenta de este hecho y obrar en consecuencia.

Un caso claro de esta circunstancia puede ser el método finalizarEjecución, que sólo tiene una línea y sólo se invoca en un único sitio dentro del programa. Normalmente, no tiene sentido crear métodos tan cortos. Por tanto, no sería incorrecto eliminarlo y incorporar su código directamente allá donde se invoca (en el tratamiento del orden "FI").

```
1 public void tratarOpcio() {  
2     Scanner lector = new Scanner(System.in);  
3     String opcio = lector.nextLine();  
4     if (opcio.equalsIgnoreCase("RT")) {  
5         registroTemperaturasSemanales();  
6     } else if (opcio.equalsIgnoreCase("MJ")) {  
7         mostrarMedia();  
8     } else if (opcio.equalsIgnoreCase("DF")) {  
9         mostrarDiferencia();  
10    } else if (opcio.equalsIgnoreCase("FI")) {  
11        //Se ha borrado el método finalizarEjecución y se ha puesto su código  
        //directamente.  
12        fin = true;  
13    } else {  
14        System.out.println("Opción Incorrecta!\n");  
15    }  
16 }
```

Mejora de métodos

En la sección "Recursos del contenido" de la web dispone de un anexo donde se explica una técnica por simplificar el código de algunos métodos con estructuras de selección largas.

Si al final del proceso aparece algún método muy largo o complejo, esto puede significar lo contrario del caso anterior: que no se ha descompuesto suficientemente el problema. Puede valer la pena volver a aplicar el proceso de descomposición a posteriori, dividiendo éste método en otros. Aunque puede parecer que, una vez el programa ya funciona, no vale la pena refinar el proceso de descomposición (al fin y al cabo, su propósito era simplificar el proceso de creación, que ya ha finalizado), piense que otra ventaja del diseño descendente es facilitar la legibilidad de su código. En el mundo de la programación tampoco debe darle pereza de ser pulidos y ordenados (con el suyo código).

Ahora bien, a veces el proceso ha sido correcto y simplemente la cantidad de líneas de código necesarias para llevar a cabo la tarea establecida es realmente grande. En casos como éstos, igualmente, es interesante repasar si el código puede mejorarse simplificándolo, buscando algún conjunto de código alternativo que lo haga más corto. Evidentemente, esto no siempre es posible, pero vale la pena hacer un pensamiento ahora que ya tiene un programa que funciona.

1.4 Solución de los retos propuestos

Reto 1

```
1 importe java.util.Scanner;
2 public class OrdenarDescendente {
3     int[] listaEnters = new int[10];
4     public static void main (String[] args) {
5         OrdenarDescendente programa = new OrdenarDescendente();
6         programa.inicio();
7     }
8     public void inicio() {
9         leerLista();
10        ordenarLista();
11        mostrarLista();
12        contarMeitatMaxim();
13    }
14    //Método con las instrucciones para leer la lista.
15    public void leerLista() {
16        System.out.println("Escribe 10 valores enteros y pulsa retorno.");
17        Scanner lector = new Scanner(System.in);
18        int índice = 0;
19        while (index < listaEnters.length) {
20            if (lector.hasNextInt()) {
21                listaEnters[index] = lector.nextInt();
22                index++;
23            } else {
24                lector.next();
25            }
26        }
27        lector.nextLine();
28    }
29    //Método con las instrucciones para ordenar la lista.
30    public void ordenarLista() {
31        for (int i = 0; i < listaEnters.length - 1; i++) {
32            for(int j = i + 1; j < listaEnters.length; j++) {
33                //La posición tratada tiene un valor más alto que el de la búsqueda... Los
34                //intercambiamos.
35                if (listaEnters[i] > listaEnters[j]) {
36                    //Para intercambiar valores es necesaria una variable auxiliar
37                    int cambio = listaEnters[i];
38                    listaEnters[i] = listaEnters[j];
39                    listaEnters[j] = cambio;
40                }
41            }
42        }
43    }
44    //Método con las instrucciones para mostrar la lista por pantalla.
45    public void mostrarLista() {
46        System.out.print("El array ordenado es: [ ");
47        for (int i = 0; i < listaEnters.length; i++) {
48            System.out.print(listaEnters[i] +
49                             " ");
50        }
51        System.out.println("");
52    }
53    //Nuevo método para resolver el nuevo subproblema
54    public void contarMeitatMaxim() {
55        int valorMaximo = listaEnters[listaEnters.length - 1] / 2;
56        int y = 0;
57        while ((listaEnters[i] < valorMaxim)&&(y < listaEnters.length)) {
58            i++;
59        }
60        System.out.println("El número de valores inferiores a la mitad del máximo es
61        + i);
62    }
63 }
```

Reto 2

```
1 ...
2 //Métodos asociados al tercer nivel de descomposición
3 public void leerTemperaturasTeclado() {
4 }
5 public void incrementarData() {
6 }
7 public void mostrarData() {
8 }
9 public void calculaMedia() {
10 }
11 public void calculaDiferencia() {
12 }
13 ...
```

Reto 3

```
1 public void calculaDiferencia() {
2     float maxima = temperaturas[0];
3     float minima = temperaturas[0];
4     for(int i = 1; i < numTemperaturas; i++) {
5         if (temperaturas[i] < minima) {
6             minima = temperaturas[i];
7         }
8         if (temperaturas[i] > maxima) {
9             maxima = temperaturas[i];
10        }
11    }
12    System.out.print((maxima - minima));
13 }
```

Reto 4

```
1 public void mostrarDiferencia() {
2     if (numTemperaturas > 0) {
3         System.out.print("\nHasta la fecha ");
4         mostrarData();
5         System.out.print(" la diferencia máxima ha sido de ");
6         calculaDiferencia();
7         System.out.println(" grados.");
8     } else {
9         System.out.println("\nNo hay temperaturas registradas.");
10    }
11 }
```

2. Parametrización de métodos

El objetivo principal del diseño descendente es ofrecer una metodología que le permita plantear la creación de un programa de una manera muy parecida a cómo lo haría con cualquier otro problema de la vida real: dividiendo problemas complejos en otros más simples y fáciles de seguir. Una vez realizada esta descomposición, los lenguajes ofrecen un mecanismo para asociar a cada subproblema un bloque de código concreto. Java, se trata métodos. El beneficio de usar métodos, en la medida correcta, es la generación de código más fácil de entender y donde puede ser posible reutilizar algunos bloques en diferentes partes, sin tener que repetir el código escrito.

Si bien la identificación de subproblemas de acuerdo a distintos niveles de complejidad es suficiente para poder descomponer el programa en pedazos de código con objetivos parciales diferenciados, y hacerlo más fácil de seguir, no es suficiente para ofrecer un alto grado de reusabilidad. En la mayoría de casos, el programa resultante dividido con métodos es directamente equivalente a trocear un programa donde sólo existe el método principal. Esto se debe a que tal y como ha usado los métodos hasta ahora, éstos servían para manipular un conjunto de variables, a menudo globales, muy específico.

En realidad, a la hora de plantear qué subproblemas existen, y su declaración asociada en forma de método al código, aparte de su nombre y qué hacen, también es posible definir ciertos aspectos vinculados a los datos que deben tratar: sus parámetros de entrada y salida.

Un parámetro es un identificador usado en la descripción de un proceso, cuyo valor en realidad puede variar por diferentes aplicaciones de este proceso.

La aparición de bloques de código repetido o casi igual indica un mal estilo de programación.

2.1 Parámetros de entrada

Los parámetros de entrada son una herramienta muy útil y extensamente usada a la hora de definir con mayor detalle las características de los subproblemas resultantes de una descomposición usando diseño descendente. Para empezar, y como resumen breve de este concepto, puede quedarse con la siguiente definición.

Un parámetro de entrada es un valor que se establece inmediatamente antes de seguir un proceso, por lo que indica los datos que debe tratar o que modifica su comportamiento.

2.1.1 Motivación: definición de problemas similares

Si se descompone hasta dos niveles el problema de cocinar unos fideos japoneses vegetales, podrían encontrarse los siguientes subproblemas:

1. Recopilar ingredientes

- (a) Comprar en el supermercado
- (b) Disponerlos sobre el mármol

2. Cocinar tallarines

- (a) Preparar agua
- (b) Hervir tallarines
- (c) Escurrir tallarines
- (d) Dejarlas preparadas

3. Cocinar zanahorias

- (a) Cortar zanahorias
- (b) Freír zanahorias
- (c) Dejar zanahorias preparadas

4. Cocinar cebollas

- (a) Cortar cebollas
- (b) Freír cebollas
- (c) Dejar cebollas preparadas

5. Preparación final

- (a) Mezclar ingredientes con salsa yakitori
- (b) Saltar ingredientes
- (c) Preparar sartén para saltar
- (d) Cocinar removiendo ingredientes
- (e) Dejar listo para servir

Partiendo de este planteamiento, la figura 2.1 muestra la división ya más detallada, hasta un tercer nivel, de los procesos de freír cebollas y zanahorias, usados en lugares bien distintos (3 y 4). En esta figura están remarcadas mediante cuadros las diferencias entre ambos.

Figura 2. 1. Diferencias entre dos problemas similares.



Ya a simple vista puede verse que, a pesar de poder descomponerse hasta en seis nuevos subproblemas, se trata de dos procedimientos muy parecidos. De hecho, si analiza con detalle ambos casos, se dará cuenta de que sólo cambia el alimento que hay que freír, pero los subproblemas a ir resolviendo son exactamente los mismos en todos los casos. De hecho, si tuviera una cierta visión de futuro, podría llegar a darse cuenta de que este escenario se aplicaría incluso en descomposiciones de otros procedimientos similares que no necesita usar aquí, como, por ejemplo, hacer patatas fritas. La única condición es que, cuando hable de freír, siempre debe estar refiriéndose a alimentos,

por supuesto.

Dicho en palabras informáticas: dados estos dos métodos, sólo varía el valor de los datos a transformar, que siempre serán de idéntico tipo. Pero el algoritmo a usar es el mismo.

La consecuencia de esta observación es que sería mucho más cómodo definir una serie de pasos de forma genérica. Por ejemplo, nombrar el proceso freír alimento, donde sólo se usa el término alimento dentro de la descripción de sus pasos para referirse a lo que se quiere freír. Entonces, siempre que se quiere freír un alimento concreto, en las explicaciones se reemplaza donde pone alimento por lo que realmente se quiere freír (cebollas, zanahorias, patatas, etc.) y se aplica el proceso tal y como está escrito, sin tener que cambiar nada. Una única descripción sirve para freír cualquier cosa, no es necesario describir un caso diferente por cada alimento.

Pues bien, si hiciera esto, el término alimento encajaría con la definición de parámetro de entrada. Su valor se establece antes de empezar el proceso, y según éste, condiciona lo que hay que tratar (en este caso, freír).

Un hecho importante en un caso como éste es que el número de parámetros de entrada permitidos no tiene por qué estar limitado a uno. Podría definir tantos como le hagan falta por cada diferencia existente, o por cada dato adicional que pueda variar según la situación y que hay que tener en cuenta para realizarlo. Por ejemplo, suponga que desea prever la posibilidad de freír con diferentes cantidades de aceite. O sea, solucionar el problema "freír alimento con cierta cantidad de aceite". En ese caso, se podrían

usar dos parámetros de entrada: el alimento y la cantidad de aceite. Los pasos a seguir seguirían siendo casi iguales, salvo que la descripción sería genérica tanto a la hora de referirse al alimento que hay que freír, como en cuanto a aceite se debe poner en la sartén. Así pues, la descripción del proceso sería independiente tanto en cuanto a qué freír como en cuanto a la cantidad de aceite a usar. Los valores concretos a aplicar dependerían de cada situación dada a la hora de freír:

- Freír patatas con 400 cl de aceite: (alimento = patatas, cantidad = 400 cl).
- Freír cebollas con 200 cl de aceite: (alimento = cebollas, cantidad = 200 cl).
- Freír escalopes con 400 cl de aceite: (alimento = escalopes, cantidad = 400 cl).
- Etc.

Es muy importante elegir correctamente el nombre de los subproblemas, para que sean descriptivos.

Por tanto, una vez tiene dividido un problema general en subproblemas, habrá que estudiar cuáles de ellos son suficientemente parecidos para, en realidad, poder considerar que se trata del mismo mediante la aplicación de parámetros de entrada. Normalmente, si el nombre que se ha puesto en los subproblemas es adecuado, esto es fácil de detectar, ya que tendrán muy similares, o incluso idénticos. Puede suceder que las divisiones en subproblemas no sean exactamente idénticas como en este ejemplo, pero entonces valdrá la pena que revise la descomposición y reflexione sobre si, en realidad, debería ser igual.

A medida que tenga experiencia, no necesitará ni siquiera esperar al final y seguro que irá detectando estas semejanzas sobre la marcha.

2.1.2 Declaración y uso de métodos con parámetros de entrada

De la misma forma que esto ahorra mucho espacio en un libro de cocina, aplicar este sistema al aplicar diseño descendente para descomponer un programa también es muy útil. Esto significa que este hecho también debe poder preverse en la declaración y codificación de un método. Por suerte, todos los lenguajes de programación que permiten declarar funciones o métodos contemplan esta posibilidad.

Declaración

La lista de parámetros de un método se escribe entre los paréntesis en su declaración. Esta lista toma una estructura parecida a realizar una serie de declaraciones de variables separadas por comas. La sintaxis exacta sería la siguiente:

```
1 public void nombreMétodo (tipoParam1 numParam1, tipoParam2 nombreParam2, etc.) {  
2     //Código  
3     ...  
4 }
```

La declaración de parámetros de entrada no será obligatoria. Si no necesita parámetros de entrada por un método, no es necesario poner nada entre los paréntesis, tal y como ha hecho hasta

ahora. Tampoco existe límite en el número de parámetros, pero normalmente es mejor no excederse. De hecho, un alto número de parámetros en el método asociado a un subproblema puede significar que no se ha descompuesto suficientemente.

Una vez que un método tiene definidos una serie de parámetros de entrada, estos se consideran variables declaradas, con ámbito dentro de todo su código. Estas variables pueden ser usadas como cualquier otra en su código. Ahora bien, tienen una peculiaridad, y es que de entrada no se les asigna ningún valor inicial, ya que vendrá dado según se invoque el método, como verá a continuación.

El método principal (main) se declara con un parámetro de entrada: un array de String.

```
1 //El método mostrarMaxim tiene dos parámetros de entrada, de tipo enter public void mostrarMaxim(int a, int b) {  
2  
3     ...  
4 }
```

Únicamente en cuanto a variables disponibles dentro del bloque de código del método, se puede considerar que este código sería equivalente a realizar:

```
1 public void mostrarMaxim() {  
2     //Se inicializan con un valor inicial de una manera especial que ya verá pronto  
3  
4     int a; int  
5     b;  
6     ...  
7 }
```

Invocación

Cuando un método se ha declarado con parámetros de entrada, deben tenerse en cuenta durante su invocación. No basta con poner sólo el nombre del método como hasta ahora. Por cada parámetro es necesario asignar un valor, exactamente en el mismo orden en que se han declarado en la lista. Este valor puede ser el resultado de cualquier expresión: un literal, una variable o una expresión más compleja. En cualquier caso, el tipo resultante debe coincidir con el del parámetro que ocupa el mismo orden.

Sintácticamente, esto se hace poniendo los valores entre los paréntesis de la invocación, separados por comas:

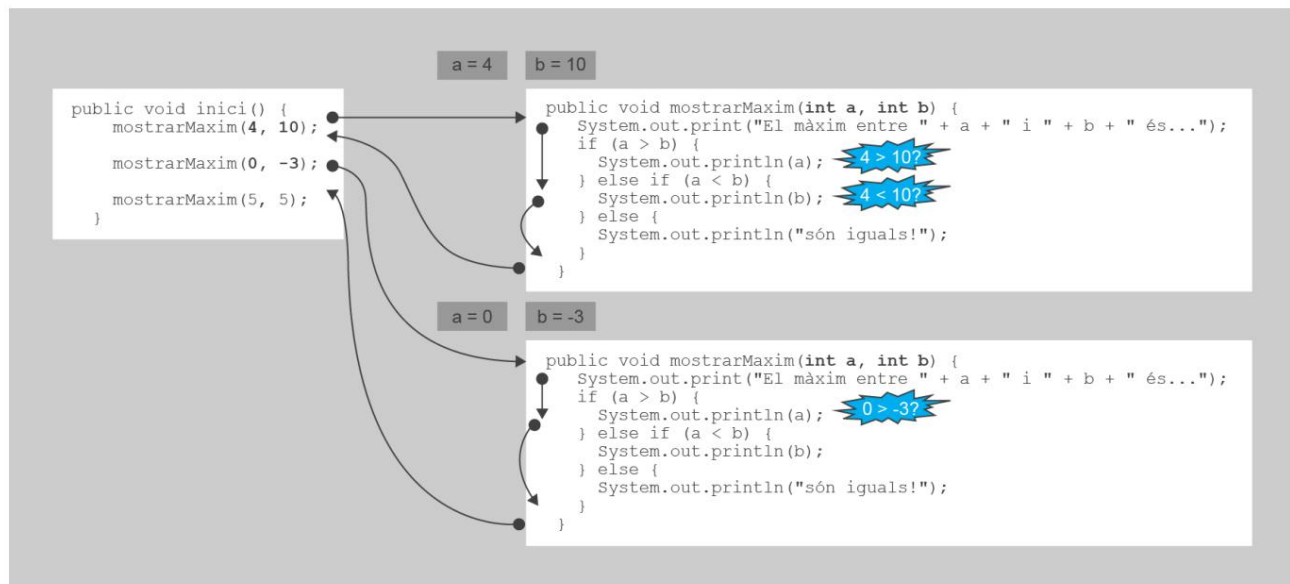
```
1 nombreMetodo(valor1, valor2, etc.);
```

Cuando se invoca un método con un conjunto de parámetros de entrada declarados, lo primero que hace el programa es mirar la lista de parámetros y declarar, implícitamente en el código del método, tantas variables como parámetros de entrada, con los mismos nombres. Entonces toma cada valor especificado en la invocación y copia, uno por uno y en el mismo orden, estos valores dentro de las variables creadas a partir de la lista de parámetros.

La mejor manera de tener una primera idea de cómo funciona todo es con un ejemplo sencillo. El siguiente código sirve para ilustrar el comportamiento de un método con parámetros de entrada. No se trata de un programa complejo resultado de aplicar estrictamente diseño descendente, sólo sirve para probar cómo funciona. Compruebe lo que hace en su entorno de trabajo.

La figura 2.2, a continuación, muestra un esquema de su flujo de control y qué está pasando a medida que se ejecuten las instrucciones del programa. Aunque el código del método `mostrarMaxim` se lista dos veces, sólo es por claridad, para diferenciar las ejecuciones por cada invocación. En el código fuente original sólo es necesario definirlo una vez. Precisamente, se trata de un caso de reaprovechamiento de código. Fíjese que la forma en que se definen los parámetros en la invocación del método sirve para dar valor a las variables `a` y `b` en el método.

Figura 2. Flujo de control al llamar un método y asignación de valores a sus parámetros



```

1 public class MaximParametroEntrada {
2
3     public static void main (String[] args) {
4         MaximParametroEntrada programa = new MaximParametroEntrada();
5         programa.inicio();
6     }
7     public void inicio() {
8         mostrarMaximo(4, 10);
9         mostrarMaximo(0, -3);
10        mostrarMaximo(5, 5);
11    }
12    //Tiene dos parámetros de entrada, de tipo entero
13    //Su valor inicial depende de los valores asignados a cada invocación
14    public void mostrarMaximo(int a, int b) {
15        System.out.print("El máximo entre if (a > b) {      " + a + " y " + b + " es...");
16
17        System.out.println(a);
18    } else if (a < b) {
19        System.out.println(b);
20    } else {
21        System.out.println("¡son iguales!");
22    }
23 }

```

En este ejemplo los valores usados al invocar el método `mostrarMaxim` expresado mediante literales, pero también podrían usarse sin problemas variables o expresiones, tal y como muestra el código listado a continuación. En este caso, las expresiones se evalúan antes de proceder a ejecutar las instrucciones del método, de modo que sea posible asignar correctamente el valor resultante a cada parámetro de entrada. Pruebe que hace exactamente lo mismo que el ejemplo anterior.

```
1 public class MaximParametroEntrada {
2     public static void main (String[] args) {
3         MaximParametroEntrada programa = new MaximParametroEntrada();
4         programa.inicio();
5     }
6     public void inicio() {
7         //Usamos literales
8         mostrarMaximo(4, 10);
9         //Usamos variables
10        int y = 0;
11        int j = -3;
12        mostrarMaximo(y, j);
13        //Usamos expresiones, con literales o variables
14        mostrarMaximo(2 + 3, y + 8);
15    }
16    //El resto del código es igual...
17 }
```

Reto 1. Haga un programa que llame varias veces un método con un único parámetro de tipo entero. Este método escribe por pantalla tantos símbolos “*” el valor del parámetro como el parámetro.

2.1.3 Manipulación de los parámetros de entrada

Una vez que se empieza a ejecutar el código del método, las variables representadas por los sus parámetros de entrada pueden ser usadas como cualquier variable declarada en la primera línea del programa: sus valores pueden ser leídos directamente, pueden usarse dentro de expresiones más o menos complejas, tienen un ámbito igual método, etc. Incluso pueden verse modificado su valor inicial. Ahora bien, debe tener claro que los valores que contienen son copias del original. Por tanto, el valor usado en la invocación nunca se verá alterado.

Por ejemplo, pruebe el siguiente código y vea como la variable *i*, usada en la invocación del código, no se ve alterada en ningún momento. Dentro del parámetro de entrada a sólo hay una copia del valor de *i*. No se trata de una asociación directa.

```
1 //Modifique el valor de un parámetro. ¿Afecta a la variable original?
2 public class ModificaParámetro {
3     public static void main (String[] args) {
4         ModificaParámetro programa = new ModificaParámetro();
5         programa.inicio();
6     }
7     public void inicio() {
8         int y = 10;
9         System.out.println("Antes de llamar al método \"y\" vale " + y);
10
11        System.out.println("Después de llamar al método \"y\" vale " + y);
12    }
13    //Tiene un único parámetro de entrada, de tipo entero
14    public void modificarParametro(int a) {
15        //Ahora hay una variable "a" declarada.
16        //Su valor depende de cómo se ha invocado el método.
17        a = 0;
18        System.out.println("Ha modificado el valor en " + a);
19    }
20 }
```

Lo que ha sucedido sería equivalente a haber hecho:

```
1 //Se dispone de una variable
2 int i = 10;
3 //Se crea una copia de la variable
4 int a = i;
5 //Se modifica la copia
6 a = 0;
7 //¿Cuánto vale y? y todavía vale 10
```

Sin embargo, en el lenguaje Java esta norma sólo se cumple estrictamente cuando se trata de tipos primitivos. Cuando el parámetro es un array, sí que se modifica el valor. Si hace memoria, esto es una consecuencia lógica de sus propiedades cuando se copian variables de tipo array. En ese caso, no se disponía realmente de dos réplicas idénticas, sino de dos variables distintas a partir de las cuales acceder exactamente a los mismos datos.

Compruebe este hecho mediante la ejecución del siguiente ejemplo:

```
1 //Modifique el valor de un parámetro. ¿Afecta a la variable original?
2 public class ModificaParametroCompuesto {
3     public static void main (String[] args) {
4         ModificaParametroCompost programa = new ModificaParametroCompost();
5         programa.inicio();
6     }
7     public void inicio() {
8         int[] y = {1, 2, 3, 4, 5};
9         System.out.println("Antes de llamar al método \"y[3]\" vale " modificarParametro(i);          + y[3]);
10
11         System.out.println("Después de llamar al método \"y[3]\" vale "          + y[3]);
12     }
13     //Tiene un único parámetro de entrada, de tipo entero
14     public void modificarParametro(int[] a) {
15         //Ahora hay una variable "a" declarada.
16         //Su valor depende de cómo se ha invocado el método.
17         a[3] = 0;
18         System.out.println("Hemos modificado el valor en          " + a[3]);
19     }
20 }
```

Lo que ha sucedido ahora sería equivalente a haber hecho:

```
1 //Se dispone de una variable de tipo array
2 int i[] = {1, 2, 3, 4, 5};
3 //Se crea una copia de la variable array
4 int a[] = i;
5 //Se modifica la copia
6 a[3] = 0;
7 //¿Cuánto vale y[3]? y[3] vale 0!
```

De hecho, esta circunstancia se da con cualquier tipo de dato compuesto del Java. La única excepción son las cadenas de texto (variables de la clase String). Su comportamiento es igual al de los tipos primitivos, ya que, recuerde, tienen la propiedad de ser inmutables. No es posible modificar su contenido.

Reto 2. Haga un programa llamado ModificarParametreString que compruebe que, efectivamente, modificar el valor de un parámetro de entrada de la clase String no tiene ningún efecto sobre el valor original.

2.2 Parámetros de salida

Como en el caso de los parámetros de entrada, se partirá de una definición ya partir de ahí verá cuál es la utilidad y modo de uso de este tipo de parámetros.

Un parámetro de salida indica un resultado final obtenido después de realizar un proceso determinado.

2.2.1 Motivación: Definición de problemas que generan un resultado concreto

Si se fija un poco, se dará cuenta de una característica común a muchos de los casos en los que un problema, en cualquier nivel, se divide en subproblemas más simples.

A menudo, conceptualmente, la forma en que se resuelve es realizando una serie de tareas donde se obtienen resultados parciales, que al final son combinados o aprovechados dentro del proceso general para obtener el resultado final (la resolución del problema).

Revisando la descomposición de la receta de cocina, simplemente en una observación a primer nivel, podrá detectar subproblemas donde su conclusión repercute en la obtención de un resultado parcial tangible, que es manipulado por subproblemas posteriores: diferentes alimentos cocinados preparados por separado. Partiendo de estos resultados parciales, se obtiene el plato final al combinarlos todos en las tareas asociadas al último subproblema. Esta circunstancia queda resumida en la tabla 2.1.

Tabla 2. 1. Subproblemas y resultados obtenidos al resolverlos

Subproblema	¿Qué hace	resultado obtenido
1. Recopilar ingredientes	Obtiene los ingredientes base (tallarinas, cebollas, etc.)	Los diferentes alimentos a procesar
2. Cocinar tallarines	De los alimentos, procesa los tallarines	Tallarinas preparadas
3. Cocinar zanahorias	De los alimentos, procesa las zanahorias	Zanahorias preparadas
4. Cocinar cebollas	De los alimentos, procesa las cebollas	Cebollas preparadas
5. Preparación final	A partir de tallarines, zanahorias y cebollas, hace el plato final	Plato final

Más allá de una receta de cocina, seguro que puede pensar en muchos casos parecidos donde los subproblemas a resolver en realidad son un mecanismo para recopilar un conjunto de resultados parciales, necesarios antes de poder llevar a cabo la resolución del problema general.

- Para escribir un libro, es necesario recopilar sus capítulos.
- Para enviar un mensaje electrónico, es necesario obtener la dirección de destino o generar el texto a enviar.

- Para montar una máquina cualquiera, es necesario obtener sus componentes. Éstos, a su vez, también se generan a partir del montaje de piezas aún más simples.
- Para vender un producto, es necesario buscar el producto propiamente, consultar el producto precio o crear un ticket.
- Etc.



Cada método debe ser una herramienta especializada para realizar una única tarea concreta. Fuente: Joan Arnedo

Por tanto, en la definición de un subproblema también puede cobrar sentido indicar que cuando éste se resuelve, se produce un resultado concreto que podrá ser usado a posteriori por otros subproblemas. De acuerdo a la definición inicial, ese resultado sería precisamente su parámetro de salida.

Teóricamente, se podría prever una tarea que, al terminar, produzca más de un resultado, ya que ha resuelto varios problemas a la vez. Para simplificar, sólo se tratará el caso de subproblemas que producen un único resultado, y por tanto, a mucho estirar con un único parámetro de salida. Por tanto, a partir de ahora, siempre que se haga la descomposición de un problema, será necesario que la plantee de manera que esto se cumpla. Partirá del principio de que si un subproblema produce más de un resultado tangible, es que no se ha descompuesto lo suficiente todavía.

Evidentemente, también puede darse el caso de que las tareas asociadas a un subproblema no den ningún resultado tangible que deba ser manipulado a posteriori. En este caso, no habrá parámetro de salida.

2.2.2 Declaración y uso de métodos con un parámetro de salida

De nuevo, dentro de la declaración de un método también es posible realizar una traslación directa de esta propiedad para codificar la resolución de un subproblema.

Hablando ya en términos de métodos dentro de sus programas, un método con un parámetro de salida explícita que servirá para generar un resultado, con un valor concreto.

Declaración

Dado que un programa genera y trata datos, lo primero que se debe decidir es a qué tipo de datos pertenecerá el valor de este resultado. Una vez decidido, la sintaxis para declarar un método con un parámetro de salida es la siguiente:

```
1 public tipoParamSalida nombreMétodo (listaParamEntrada) {  
2     //Código  
3     ...  
4 }
```

El tipo de parámetro de salida puede ser cualquiera, identificado por la palabra clave correspondiente, como en el caso de una variable. Si se quiere indicar que no hay ninguna

parámetro de salida, se usa la palabra reservada void, tal y como ha usado hasta ahora en todos los métodos antes de introducir ese concepto.

El código del método será el que corresponda para obtener el resultado esperado, de acuerdo a su propósito. En este aspecto, nada cambia respecto a cómo habéis programado métodos hasta ahora. Ahora bien, dentro de este código habrá que decidir, entre todos los datos que trata internamente, cuál se corresponde al resultado final. Una vez lo ha establecido, la sentencia que indica que ya ha terminado la tarea a realizar y dispone del resultado es la siguiente:

```
1 return resultado;
```

De hecho, en cuanto se invoca la sentencia `return resultado`, la ejecución del método termina inmediatamente y ya no se ejecuta ninguna instrucción más.

Ésta normalmente se escribiría la última de todas, al final del bloque de código asociado al método. Donde pone `resultado` se puede usar cualquier expresión que evalúe el tipo declarado por el parámetro de salida: un literal, una variable, o una combinación cualquiera de operandos y operadores. Si se quiere ir despacio, normalmente lo más recomendable es que guarde el resultado en una variable y haga `return` de ésta (`return nombreVariable;`).

Escribir la sentencia `return` es indispensable, ya que si no se pone habiendo declarado un parámetro de salida, habrá un error de compilación.

Dado el nombre de la sentencia usada, a menudo en el parámetro de salida se le llama también valor de retorno de un método. A partir de ahora se usará también esta nomenclatura indistintamente.

Evidentemente, la mejor manera de ver el funcionamiento de todo ello es con un ejemplo sencillo. Suponga que desea definir un método que sirve para leer correctamente un único valor entero desde el teclado, de modo que lo haga definiendo un parámetro de salida. Su código podría ser el siguiente (Lea atentamente los comentarios del código, ya que indican el conjunto de decisiones que hay que ir haciendo por codificarlo correctamente.):

```
1 //1. ¿Qué tipo de valor genera? Un entero (int)
2 public int leerEnterTeclado() {
3     //2. Se hace el código que lee un único entero del teclado, como siempre se ha hecho
4     //No cambia absolutamente nada...
5     Scanner lector = new Scanner(System.in);
6     int enteroLeído = 0;
7     boolean leído = false;
8     while (!leído) {
9         leído = lector.hasNextInt();
10        if (leído) {
11            enterLeído = lector.nextInt();
12        } else {
13            System.out.println("Esto no es un entero.");
14            lector.next();
15        }
16    }
17    lector.nextLine();
18    //3. Una vez hecho, ¿qué variable tiene el resultado? "enteroLeído"
19    //4. Hay que hacer "retorno" sobre él
20    return enteroLeído;
21 }
```

Invocación

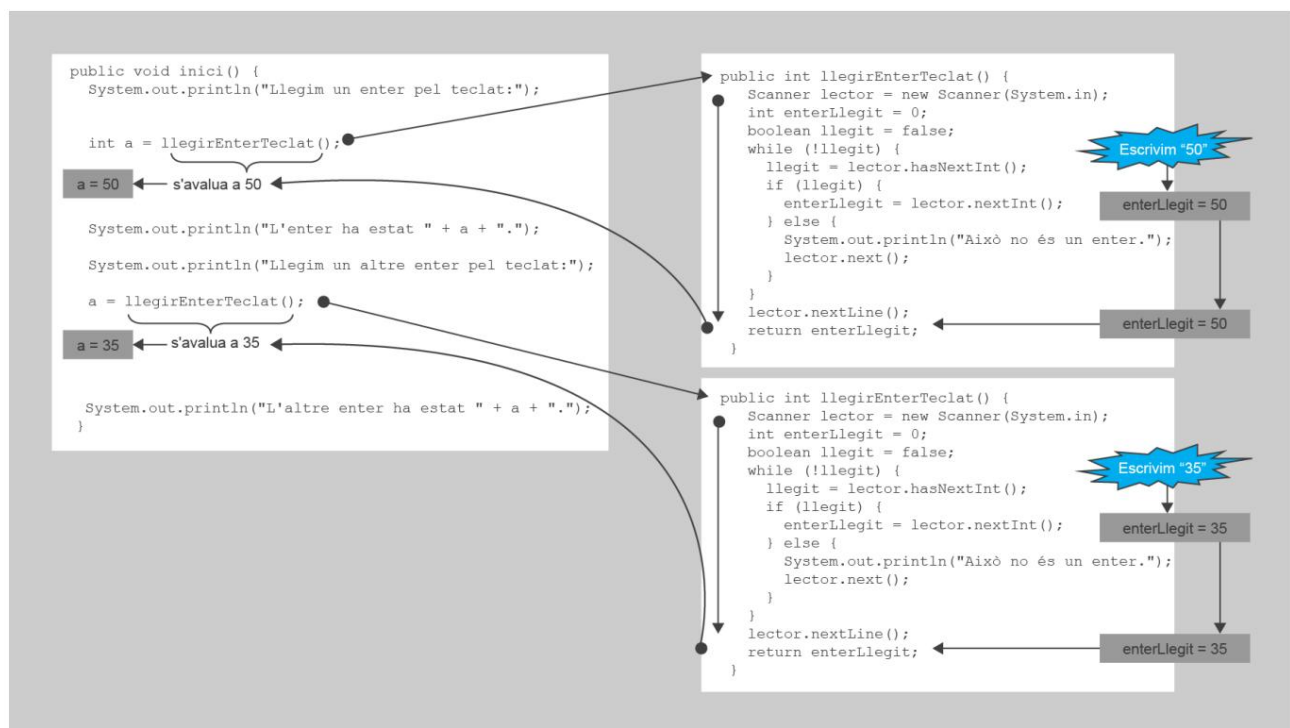
Como en toda expresión, si desea obtener el resultado calculado por el método por consultarlo a posteriori, será necesario que lo guarde en una variable.

El aspecto más importante cuando existen métodos con un valor de retorno definido dentro del su código es la invocación, ya que existe una variación sustancial respecto al modelo visto hasta el momento. La sintaxis para invocar el método es exactamente la misma, poniendo el nombre y la lista de valores para los parámetros de entrada (si los hubiere). Al llevarse a cabo la invocación, el flujo de ejecución salta también a las instrucciones definidas dentro de su bloque de código. Ahora bien, lo que cambia es el comportamiento del método, puesto que pasa a considerarse una expresión. Éste evaluará el valor resultante para a esa invocación.

Para ver este proceso, fíjese en el siguiente código, en el que se invoca el método definido anteriormente. Compílo y ejecútelo, para ver qué sucede.

La figura 2.3 muestra un esquema de cuál es el flujo de control asociado a la primera invocación y qué está sucediendo en cada momento. Por claridad en la figura, el código del método leerEnterTeclat está repetido por las diferentes invocaciones, pero, de nuevo, tenga en cuenta que en el código fuente sólo aparece una única vez. Simplemente sirve para mostrar que su resultado puede variar por diferentes invocaciones.

Figura 2.3. Flujo de control al llamar un método y evaluación del método usando el valor de retorno.



```

1 import java.util.Scanner;
2 public class LeerEnteros {
3     public static void main (String[] args) {
4         LeerEnteros programa = new LeerEnteros();
5         programa.inicio();
6     }
7     public void inicio() {
8         System.out.println("Lea un entero por el teclado:");
9         int a = leerEnterTeclado();
10        System.out.println("El entero ha sido " + a + ".");
11        System.out.println("Lea otro entero por el teclado:");
12        a = leerEnterTeclado();
13        System.out.println("El otro entero ha sido " + a + ".");

```

```
14  
15     } public int leerEnterTeclado() {  
16         //El código no varía respecto al ejemplo anterior //...  
17  
18     }  
19 }
```

Si hace memoria, este modelo de invocación es exactamente igual a la invocación de métodos sobre una variable de tipo String. Esto se debe a que los métodos definidos por String están declarados con valores de retorno.

Reto 3. Modifique el ejemplo de modo que en lugar de usar enteros, funcione con reales (double). Asegúrese de su correcto funcionamiento introduciendo números con decimales.

2.3 Cuando declarar parámetros de entrada o salida

Como se ha ido mostrando, el uso de parámetros, tanto de entrada como de salida, resulta especialmente útil para convertir problemas muy parecidos en uno mismo y hacer que el proceso de descomposición de un programa sea conceptualmente similar al de un problema de ámbito general. Ahora bien, en realidad, su uso no debería limitarse sólo a este escenario, sino que debería ser generalizado a cualquier método. De hecho, el uso de parámetros de entrada y salida no se trata de un caso especial diferenciado en la declaración de parámetros, sino lo habitual cuando se usan dentro del código de un programa.

Por otra parte, hasta ahora, las variables globales se han presentado como mecanismo para compartir o traspasar datos entre métodos de distintos niveles de descomposición. Pero en realidad también es lo más normal usar parámetros como mecanismo para esta tarea, reemplazando mayoritariamente el uso de variables globales. A partir de ahora, se considerará que es mejor evitar su uso, excepto en casos muy concretos. Por tanto, ante la pregunta “¿cuándo declarar parámetros de entrada y salida en un método?”, la respuesta es “siempre que se pueda y tenga sentido”, independientemente de que un método en realidad se invoque una sola vez o muchas dentro de su programa.

2.3.1 Mejorando la legibilidad de código con parámetros

Una parte de la utilidad de los métodos es mejorar la comprensión del código de los programas, por lo que no es necesario ir siguiendo y entendiendo las instrucciones en un blog demasiado largo, y de una sola vez. Si los identificadores de los métodos se han escogido con criterio, el código es más fácil de seguir. Este seguimiento es aún más sencillo si se usan parámetros, ya que no dejan ninguna duda sobre los datos que está tratando un método dado.

Por ejemplo, fíjese en el código que se muestra a continuación y plantéese sinceramente las siguientes preguntas:

- En esta invocación, ¿qué valores se están tratando en un método?
- ¿Cuál es el resultado para este caso?
- Si quisiera mostrarlo por pantalla, ¿en qué variable se encuentra?

```
1 //Son variables globales 2 y = 4; j = 8; 4 k = 12;  
3  
5 ...  
6 unMetodo();
```

De hecho, es imposible contestar a las preguntas sin saber exactamente cómo funciona este método. Debería ir inmediatamente a su código fuente e inspeccionarlo detalladamente.

En cambio, compare con este otro código:

```
1 //No son variables globales 2 int i = 4; 3 int j = 8; 4 int k  
= 12;  
5 ...  
6 k = unMetodo(i, j);
```

Sin siquiera saber qué hace el método, no es necesario saber cuál es su código para responder. Sin lugar a dudas, los valores que se están tratando son 4 y 8, y el resultado estará almacenado en la variable k. Por tanto, elegir usar parámetros en lugar de variables globales es especialmente positivo para la legibilidad del código.

2.3.2 El principio de ocultación / encapsulación

Un motivo para usar de forma generalizada parámetros de entrada y salida en sus métodos y limitar el uso de las variables globales es garantizar uno de los principios más importantes dentro de la programación.

El principio de encapsulación/ocultación se define formalmente como la segregación de los elementos en los que se descompone un problema general, por lo que se separa su interfaz contractual de su implementación.

Dicho en otras palabras, un método debe realizar una única tarea muy concreta, de forma autocontenida, y para poder ser invocado no es necesario saber realmente cómo funciona, sólo qué hace. Alguien que no haya programado el método debe ser capaz de usarlo sin tener que saber cuál es su código o qué variables se están usando en su interior. Si para poder invocar satisfactoriamente un método se requiere saber parte del suyo

código, no se está siguiendo este principio. De hecho, hasta cierto punto, este principio está ligado a los motivos anteriormente expuestos para evitar el uso de variables globales.

Con el fin de seguir fielmente el principio de encapsulación/ocultación, puede ser útil establecer unas pautas generales de comportamiento de los métodos cuando se quieren considerar en forma parametrizada. El aspecto principal para alcanzar este objetivo es darse cuenta de que, aunque a la hora de declarar un método los parámetros de entrada y el de salida son aspectos totalmente independientes sintácticamente, en la mayoría de casos están muy relacionados en su código (figura 2.4).

Normalmente, los métodos son utilizados como mecanismo para encapsular bloques de código autocontenidos que realizan cálculos o transformaciones sobre los valores de los parámetros de entrada, a partir de los cuales generar un resultado final concreto. No es casualidad que en muchos lenguajes a los métodos se les llame “funciones”, ya que se comportan exactamente como funciones matemáticas: partiendo de unos valores de entrada dados, se aplica una fórmula (en este caso, un proceso) mediante la cual se calcula el resultado.

Ejemplos de problemas típicos que ha visto y que cumplen la condición de que, dados unos valores de entrada, es necesario calcular un resultado concreto, pueden ser:

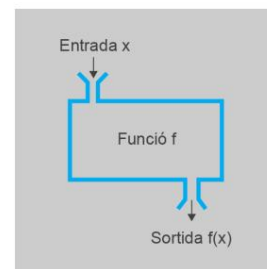
- El máximo (o el mínimo) entre un conjunto de valores (por ejemplo, un array).
- La media aritmética de los valores en un array.
- Dado un valor de nota, transformarla en cadena de texto (por ejemplo, 5 = Aprobado).
- Saber los días que tiene un mes dado.
- Buscar la posición de un valor concreto dentro de un array (si lo hubiere).

2.3.3 Ejemplos de métodos parametrizados

A modo de ejemplo, a continuación se muestra la traducción de algunos de los blogs de código asociados a problemas con los que ha trabajado en métodos con parámetros de entrada y un valor de retorno. Dentro del código encontrará en formato comentario las explicaciones sobre qué tipo de datos tratan, y por tanto, cuáles son sus parámetros de entrada y salida.

En estos ejemplos sólo se muestra el código asociado al método que realiza cada tarea, no el programa completo, con los métodos principal (main) e inicio.

Calcular el mínimo entre dos enteros



En última instancia, los métodos suelen encapsular una transformación de los datos de entrada en datos de salida

```

1 //Parám. entrada: los valores a tratar son dos enteros //Parám. salida: el mínimo entre los dos, un entero público int
2  minim (int a, int b) { // "a" y "b" contienen los valores a tratar
3
4

```

```
5     int min = b;  
6     if (a < b) {  
7         min = a;  
8     }  
9     // "min" contiene el resultado  
10    return min;  
11 }
```

Para invocarlo para calcular el mínimo entre 4 y 10, y guardar el resultado en una variable llamada nada, se haría:

```
1 int res = minim(4, 10);  
2 // En este caso, "nada" valdrá 4
```

Reto 4. Usando este método, realice un programa que calcule cuál es el valor mínimo entre los valores 3, 6, 10 y 15, y lo muestre por pantalla.

Calcular lo máximo dentro de un array de reales

```
1 // Parám. entrada: el valor a procesar es un array  
2 // Parám. salida: el máximo entre todos, un real  
3 public double maxim(double[] array) {  
4     // "array" contiene el valor a tratar  
5     double max = array[0];  
6     for (int i = 1; i < array.length; i++) {  
7         if (array[i] > max) {  
8             max = array[i];  
9         }  
10    }  
11    // "max" contiene el resultado  
12    return max;  
13 }
```

Para invocarlo para calcular el máximo en el array llamado valores y guardar el resultado en una variable llamada nada, se haría:

```
1 double[] valores = {1.3, -2.1, 0, 12.0, 4.7};  
2 ...  
3 double res = maxim(valores);  
4 // En este caso, "nada" valdrá 12.0
```

Calcular la media aritmética de los valores de un array

```
1 // Parám. entrada: el valor a procesar es un array  
2 // Parám. salida: el resultado de realizar el cálculo, un real (tiene decimales)  
3 public double media(int[] array) {  
4     // "array" contiene el valor a tratar  
5     double acumulador = 0;  
6     for (int i = 0; i < array.length; i++) {  
7         acumulador = acumulador + array[i];  
8     }  
9     // El resultado es (acumulador)/(número elementos)  
10    // Se puede devolver el resultado de una expresión  
11    return acumulador/array.length;  
12 }
```

Para invocarlo para calcular la media del array valores y guardar el valor en la variable nada, se haría:

```
1 int[] valores = {1, 2, 3, 4, 5};
2 ...
3 double res = media(valores);
4 //En este caso, "nada" valdrá 3
```

Transformar una nota a texto

```
1 //Parám. entrada: el valor a procesar es un real (una nota)
2 //Parám. salida: una cadena de texto
3 public String notaAText(double nota) {
4     //"nota" contiene el valor a tratar
5     String text = null;
6     if ((nota >= 9)&&(nota <= 10)) {
7         texto = "Excelente";
8     } else if ((nota >= 6.5)&&(nota < 9)) {
9         texto = "Notable";
10    } else if ((nota >= 5)&&(nota < 6.5)) {
11        texto = "Aprobado";
12    } else if ((nota >= 0)&&(nota < 5)) {
13        texto = "Suspendido";
14    } else {
15        texto = "Nota no válida";
16    }
17    //"texto" contiene el resultado
18    return texto;
19 }
```

Para invocarlo para saber el texto que corresponde a la nota 7.8, guardar el valor en la variable textNota y mostrarlo por pantalla, se haría:

```
1 String textNota = notaAText(7.8);
2 //En este caso, "textNota" valdrá "Notable"
3 System.out.println(textNota);
```

Reto 5: haga un programa que, dado un conjunto de cinco notas parciales almacenadas en un array de reales, muestre el texto de la nota final (la media de todas cinco). Pista/Nota: tenga en cuenta que el parámetro de entrada del ejemplo media es un array de enteros. (No puede usar su código directamente, lo deberá de modificar.)

Calcular los días de un mes

```
1 public static final int[] numDias = {31, 28 30, 31}, 31, 30, 31, 30, 31, 31, 30, 31,
2     ...
3
4 //Parám. entrada: el valor a procesar es un entero (un nº de mes)
5 //Parám. salida: un entero (un núm. de días)
6 public int díasDelMes(int mes) {
7     //"mes" contiene el valor a tratar
8     int días = 0;
9     if ((mes >= 1)&&(mes <= 12)) {
10        días = numDias[mes - 1];
11    }
12    //"días" contiene el resultado
13    return días;
14 }
```

Para invocarlo para saber los días de febrero y guardarlos en la variable nada, se haría:

```
1 int res = díasDelMes(2);  
2 //En este caso, "nada" valdrá 28
```

Buscar la posición donde existe un valor concreto en un array de enteros

```
1 //Parám. entrada: los valores a procesar son el array y qué buscar (un entero)  
2 //Parám. salida: un entero (una posición del array)  
3 public int buscarValor(int[] array, int valor) {  
4     //"array" contiene el valor a tratar  
5     int posición = 0;  
6     boolean encontrado = false;  
7     while ((posición < array.length)&&(!encontrado)) {  
8         if (array[posición] == valor) {  
9             encontrado = true;  
10        }  
11        posición = posición + 1;  
12    }  
13    //¿Se ha encontrado?  
14    if (!encontrado) {  
15        posición = -1;  
16    }  
17    //"posición" contiene el resultado.  
18    //Si no se ha encontrado se hace valer -1  
19    return posición;  
20 }
```

Para invocarlo para encontrar si en alguna posición de un array llamado valores hay el valor 7 y guardar el índice de esta posición en la variable nada, se haría:

```
1 int[] valores = {1, 2, 3, 4, 5};  
2 ...  
3 int res = buscarValor(valores, 7);  
4 //En este caso, "nada" valdrá -1
```

Reto 6. Haga un programa que pregunte un número entero por el teclado. Entonces, dado un conjunto de valores dentro de un array, debe calcular cuántas veces aparece el número introducido. Use métodos tanto para preguntar el número como para buscar su número apariciones. Declare el array dónde hacer la búsqueda como una variable dentro del método inicio, inicializada con valores concretos de su elección.

2.4 Un ejemplo de diseño descendente con métodos parametrizados

Para saber cómo usar métodos basados en parámetros de entrada y salida (el más habitual cuando haga programas), a continuación verá un nuevo ejemplo de proceso completo de descomposición de un problema relativamente complejo mediante diseño descendente. Sin embargo, esta vez los subproblemas se analizarán también desde el punto de vista de qué datos son necesarios para su resolución (entrada) y qué resultado concreto producen (salida). Sin embargo, esta vez no se detallarán todos los aspectos de la descomposición. Este ejemplo sirve principalmente como hilo argumental para resaltar algunos aspectos importantes del uso de métodos dentro de un programa más complejo, como la propagación de los valores entre invocaciones de los métodos.

En este caso, el programa que se desea realizar es una versión algo más avanzada del juego de adivinar un número secreto. En este caso, se trata de adivinar una combinación de 5 letras al azar. Siempre que se realiza un intento, el programa da una pista. Por cada letra, se indica si es totalmente incorrecta (.), si se ha acertado (O), o si esa letra existe, pero no en esa posición (X). El programa se ejecuta hasta que el usuario acierta, momento en el que se da un mensaje de felicitación.

Para aclarar este funcionamiento, a continuación se da un ejemplo de ejecución:

```
1 Escribe 5 letras minúsculas: atzbq 2 La respuesta es  
[-OX--]. ¡Sigue intentándolo!  
3 Escribe 5 letras minúsculas: btymz 4 La respuesta es  
[-O-XX]. ¡Sigue intentándolo!  
5 Escribe 5 letras minúsculas: mtzoc 6 La respuesta es  
[OOX--]. ¡Sigue intentándolo!  
7 Escribe 5 letras minúsculas: mtdzp 8 La respuesta es  
[OO-OX]. ¡Sigue intentándolo!  
9 Escribe 5 letras minúsculas: mtpze 10 La respuesta es  
[OOOO-]. ¡Sigue intentándolo!  
11 Escribe 5 letras minúsculas: mtpzx 12 La respuesta es [OOOOO].  
¡Has acertado!
```

2.4.1 Descomposición de problema

De nuevo, el primer paso a dar, antes de ponerse a generar código, es sentarse un rato a pensar y hacer una propuesta sobre qué datos habrá que tratar y cómo se podría descomponer el problema.

Identificación de los datos a tratar

En un vistazo preliminar, para este problema, los datos más importantes a tratar en general son básicamente dos: la palabra secreta y la palabra que responde el usuario. También, en algún momento, habrá que generar la respuesta del programa con la pista diciendo qué letras ha acertado y controlar cuándo hay que terminar la ejecución, una vez acertado.

Primer nivel de descomposición

Antes de poder hacer nada, habrá que generar la palabra secreta de forma aleatoria. Una vez realizado, el programa se dedica a preguntar la respuesta del usuario ya continuación analizarla para generar una respuesta con la pista, o la felicitación si ha acertado. Por tanto, en un primer nivel se podría descomponer como:

1. Generar palabra secreta.
2. Preguntar respuesta.
3. Resolver respuesta.

Segundo nivel de descomposición

Para ver si es necesario un segundo nivel de descomposición, es necesario analizar si los subproblemas definidos en el primer nivel se consideran complejos, o al menos, si conceptualmente son más fáciles de entender si se vuelven a descomponer.

- Generar palabra secreta. Generar una palabra secreta puede considerarse como la generación repetidas veces de una única letra aleatoria.
- Preguntar respuesta. De entrada, leer una cadena de texto desde el teclado es un problema ampliamente tratado y que ya debería saber resolver. Por tanto, no es nada complejo.
- Resolver respuesta. Resolver la respuesta significa que, ante todo, hay que generar la pista y después ver si se corresponde con una combinación ganadora (OOOOOO) o no. Por tanto, se podría dividir en estos dos subproblemas.

1. Generar palabra secreta.

- (a) Generar letra aleatoria.
- (b) Montar palabra.

2. Preguntar respuesta.

3. Resolver respuesta.

- (a) Generar pista.
- (b) Dar respuesta.

Tercer nivel de descomposición

Nuevamente, se analizan los nuevos subproblemas para ver si es necesario continuar descomponente o no.

```
"System.nanoTime"()  
Ésta es la instrucción que  
accede al tiempo con precisión de  
nanosegundos (una  
milmillonésima de segundo, un  
segundo = 1.000.000.000  
nanosegundos).
```

- Generar letra aleatoria. Este subproblema requiere un poco de imaginación para plantearlo (y un poco de conocimiento de qué instrucciones tiene Java que le puedan ayudar). Por tanto, según se plantee, será fácil o difícil. En este ejemplo se hará de la siguiente forma. Java permite acceder a la hora actual con precisión de nanosegundos usando una instrucción especial. Esto es un tiempo tan reducido que siempre se obtendrán valores distintos por cada ejecución. Por tanto, es como generar valores al azar. Dado que ya sabe cómo, partiendo de un entero, obtener texto (por ejemplo, al convertir un número de mes en su nombre), ya se ha llegado a un problema bastante simple.
- Montar palabra. Se trataría de concatenar 5 letras para crear un String. Esta tarea puede resolverse directamente con el operador + de las cadenas de texto.

- Generar pista. Habrá que comparar la palabra secreta y la respuesta letra a letra. Esto puede hacerse con una estructura de repetición, que no es demasiado compleja, y las letras (los caracteres) pueden compararse fácilmente.
- Dar respuesta. Se trata de realizar una comparación y mostrar un mensaje por pantalla. Es claramente una estructura de selección, lo que es sencillo.

Por tanto, ya ha terminado la descomposición.

2.4.2 Implementación del segundo nivel de descomposición

Una vez establecida la descomposición a partir de la cual trabajar, ya podría establecerse una primera versión del esqueleto de la clase. Para ello, ahora sería necesaria una etapa de reflexión adicional, ya que para establecer las cabeceras de los métodos hay que tener claros cuáles serán sus parámetros de entrada y salida. Desgraciadamente, esto obliga hasta cierto punto a tener una visión general de cómo debe funcionar todo el programa y las interacciones entre métodos.

Debido a esto, en un caso como éste puede ser mejor ir despacio y sólo detenerse a pensar la declaración exacta de cada método con sus parámetros cuando corresponda, siguiendo paso a paso cada nivel de descomposición. Aparte de eso, sí es posible escribir ya los métodos main y el esqueleto del método inicio, ya que siempre siguen la misma estructura.

Normalmente, los métodos que realizan cálculos exclusivamente a partir de entrada/salida (teclado, reloj interno, etc.) no tienen parámetros de entrada.

Dada la descomposición del problema, es necesario declarar dos métodos, que se pueden decir:

- generarLetraAleatoria. Genera una letra al azar. Por tanto, el valor de retorno será un carácter. No necesita ninguna entrada para realizar este cálculo, ya que todo lo necesario se obtiene directamente de una instrucción de Java para saber los nanosegundos presentes.
- generarPista. Genera la pista que se debe mostrar por pantalla (por ejemplo, OXX.O). Esto es un texto, por lo que el valor de retorno será un String.
Para poder realizar esta tarea, es necesario comparar la respuesta del usuario (un texto) y el valor secreto (otro texto). Por tanto, se necesitan dos parámetros de entrada, también de tipo String. En el código de este método seguro que habrá que manipular las cadenas de texto.

Dado esto, ahora mismo, el código del programa podría ser así. Fíjese atentamente que la traducción de un entero aleatorio a un carácter aleatorio se hace con la ayuda de una tabla de traducción en forma de texto con todo el alfabeto (constante abc):

```
1 importe java.util.Scanner; 2 //Un programa para jugar en
el Mastermind. 3 public class MasterMind {
4     //Constantes
5     public final static char TOT_CORRECTE = 'O'; public final static
6     char MALA_POSICION = 'X';
```

```
7 public final static char INCORRECTO = '-';
8 public final static String abc = "abcdefghijklmnopqrstuvwxyz";
9 //Métodos asociados al problema general
10 //El método main no cambia respecto a la plantilla general
11 public static void main (String[] args) {
12     MasterMind programa = new MasterMind();
13     programa.inicio();
14 }
15 //Se puede declarar, pero todavía no corresponde deducir el código
16 public void inicio() {
17     //El código se hará al terminar el primer nivel de descomposición
18     //...
19 }
20 //Métodos asociados al primer nivel de descomposición
21 //Ya pensarán en terminar el segundo nivel de descomposición
22 //...
23 //Métodos asociados al segundo nivel de descomposición
24 //Parám. entrada: ninguna
25 //Parám. salida: una letra (un carácter)
26 public char generarLetraAleatoria() {
27     long nano = System.nanoTime();
28     int index = (int)(nano % abc.length());
29     return abc.charAt(index);
30 }
31 //Parám. entrada: la respuesta y el secreto a comparar (texto)
32 //Parám. salida: la pista a mostrar (un texto)
33 public String generarPista(String s, String r) {
34     String nada = "";
35     //Hay que comparar cada carácter de la solución y la respuesta
36     for (int i = 0; i < s.length(); i++) {
37         //Obtener ambos caracteres a comparar
38         char charSecret = s.charAt(i);
39         char charRespuesta = r.charAt(i);
40         if (charSecret == charRespuesta) {
41             nada = nada + TODO_CORRECTO;
42         } else if (s.indexOf(charRespuesta) != -1) {
43             //indexOf busca si un carácter concreto existe en un String
44             nada = nada + MALA_POSICION;
45         } else {
46             nada = nada + INCORRECTO;
47         }
48     }
49     return nada;
50 }
51 }
```

2.4.3 Implementación del primer nivel de descomposición

Una vez finalizado un nivel, puede subirse al siguiente. De acuerdo a la descomposición del problema general, se puede proceder a establecer los nombres de los nuevos métodos y cuáles deberían ser sus parámetros. En este caso, sin embargo, para algunos de los métodos vale la pena estudiar su implementación de forma mucho más detallada, ya que este ejemplo ha sido buscado con la intención de que aparezcan algunos aspectos interesantes cuando interactúan métodos con parámetros de entrada y/o salida.

De entrada, aquí tiene una descripción general:

- **generarPalabraSecreta.** Genera la palabra secreta, compuesta por 5 letras. Por tanto, el resultado es un texto (un valor de tipo String). Cómo que parte de cero para hacerlo, no necesita parámetros de entrada.

- `preguntarRespuesta`. Como en ejemplos anteriores, deberá ser un método que lee un texto desde el teclado y lo devuelve. Por tanto, tampoco le hacen falta parámetros de entrada y devolverá el texto leído.
- `resolverRespuesta`. Resolver una respuesta significa generar la pista, establecer si ha acertado, y mostrar un mensaje en consecuencia. Cuando se trate con más detalle ya verá cuáles deberían ser sus parámetros.

generarPalabraSecreta

Este método se basa en el uso repetido de la generación de un carácter individual para realizar una palabra de 5 letras. Por tanto, invoca internamente a `generarLetraAleatoria` ya partir de cada resultado crea su propio resultado. En parte de ello, el hecho de que un método pueda invocar, a su vez, otro método, no tiene otra peculiaridad.

```
1 //Parám. entrada: ninguna, todo se obtiene de generarLetraAleatoria
2 //Parám. salida: una palabra de 5 letras (un String)
3 public String generarPalabraSecreta() {
4     String nada = "";
5     for (int i = 0; i < 5; i++) {
6         nada = nada + generarLetraAleatoria();
7     }
8     return nada;
9 }
```

preguntarRespuesta

Escribir texto pidiendo al usuario que introduzca una palabra por el teclado y leerla en forma de cadena de texto no es excesivamente complicado, sobre todo, dado que no es necesario hacer comprobaciones de tipos, como ocurre con números (`hasNextInt`, etc.). Ahora bien, si se quiere ser estricto, debería comprobarse al menos que la respuesta sigue el formato esperado: tiene cinco letras y está formada por caracteres del alfabeto en minúscula (a...z). Ahora bien, comprobar estas dos cosas no se realiza en una o dos líneas de código.

En casos como éste, cuando se quiere realizar una comprobación de cierta complejidad sobre el formato de una lectura de datos escritos por el usuario, es especialmente útil plantearse como un nuevo subproblema. Separar todo el código asociado a la comprobación en un método aparte simplifica el código y mejora su comprensión. En este caso, el nuevo método podría llamarse `comprobarRespuesta`.

```
1 //Parám. entrada: ninguna, todo se obtiene del teclado
2 //Parám. salida: la palabra de respuesta (un String)
3 public String preguntarRespuesta() {
4     Scanner lector = new Scanner(System.in);
5     boolean lecturaOk = false;
6     String nada = null;
7     do {
8         System.out.print("Escribir" nada + " + LONG_SECRET + " letras minúsculas: ");
9         lector.next();
10        lector.nextLine();
11        lecturaOk = comprobarRespuesta(nada);
12        if (!lecturaOk) {
13            System.out.println("¡Esta respuesta no es válida!");
14        }
15    }
```

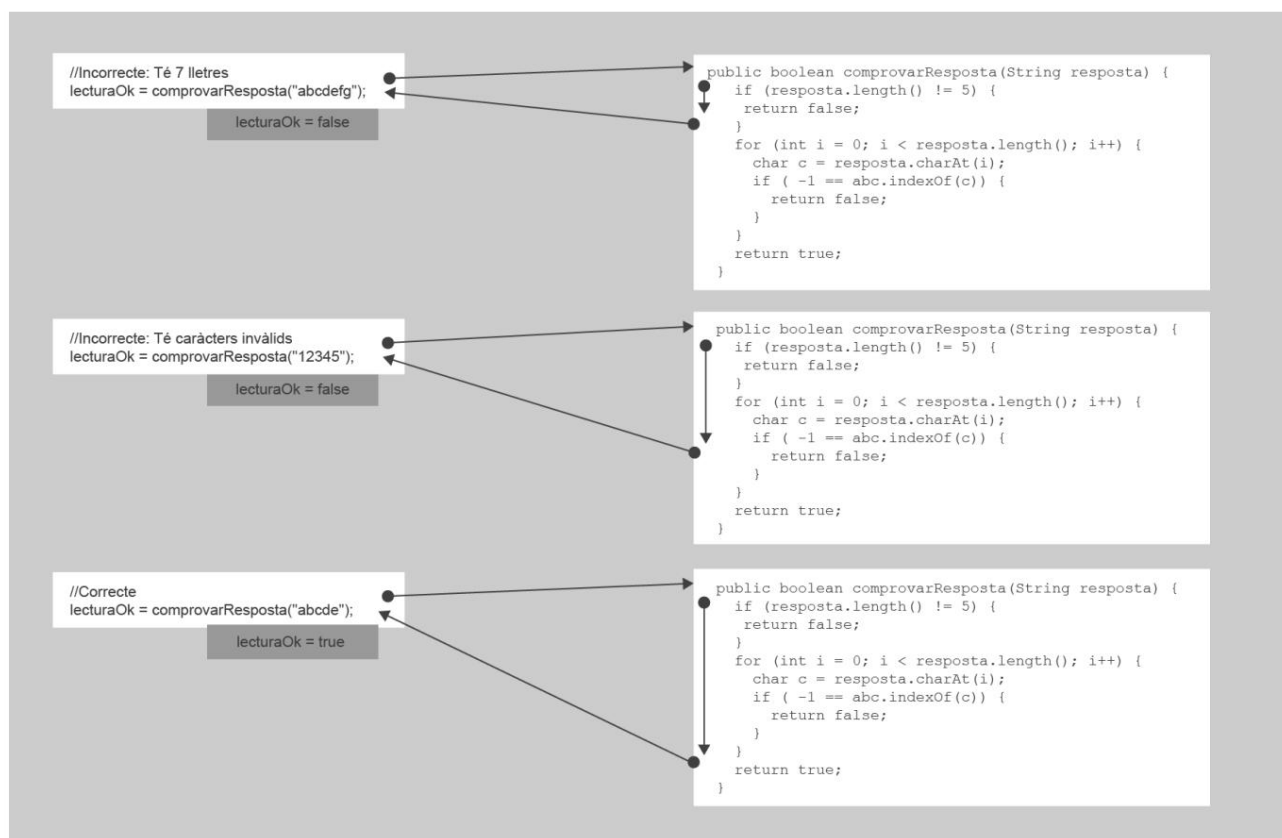
```

15     } while (!lecturaOk);
16     return nada;
17 }
18 //Este método corresponde al segundo nivel de descomposición
19 //Parám. entrada: texto a comprobar
20 //Parám. salida: si es correcto o no (un booleano. true = correcto)
21 public boolean comprobarRespuesta(String respuesta) {
22     if (respuesta.length() != 5) {
23         //Ya sabemos que no es correcto.
24         //Podemos terminar la ejecución del método inmediatamente.
25         return false;
26     }
27     for (int i = 0; i < respuesta.length(); i++) {
28         char c = respuesta.charAt(i);
29         if (-1 == abc.indexOf(c)) {
30             //Ya sabemos que no es correcto.
31             //Podemos terminar la ejecución del método inmediatamente.
32             return false;
33         }
34     }
35     //Si todo se cumple, seguro que es correcto
36     return true;
37 }

```

Este tipo de métodos que sirven para establecer si un valor se considera correcto prácticamente siempre tienen el mismo formato: la entrada son los valores en comprobar y la salida, un booleano. Hay que hacer que ésta sea true si los valores se consideran correctos y false de lo contrario (“¿Es el valor correcto? Cierto/Falso”). Entonces, con una estructura de selección que compare el valor de retorno es fácil actuar en consecuencia.

Figura 2. 4. Flujo de control del método "comprovarRespuesta" ante diferentes situaciones



Por tanto, apenas se acaba de presentar una situación donde, aunque de entrada se ha considerado que no era necesario descomponer más, a posteriori se ha establecido que es mejor hacerlo. Este hecho no es muy raro en el proceso de generación de un programa.

Si se fija, el código de comprobar Respuesta tiene una peculiaridad. Hay más de una sentencia `return`. Esto es posible, pero para entender qué está haciendo, es necesario tener presente un aspecto esencial de esta sentencia: cuando se ejecuta, el método termina inmediatamente y el valor de retorno es igual al indicado dentro de la sentencia `return`. La figura 2.4 esquematiza el funcionamiento del método de acuerdo a este comportamiento, según distintas situaciones.

La técnica
comprobar Respuesta también
es habitual y puede resultarle
muy útil para simplificar el
código de sus programas.

resolverRespuesta

Este método lo primero que hace es invocar generar pista y mostrar el valor devuelto. Ahora bien, según este valor indica que si se ha acertado (OOOOO) o no, actúa en consecuencia. Lo que hace que valga la pena estudiarlo con detalle es ver qué parámetros tiene.

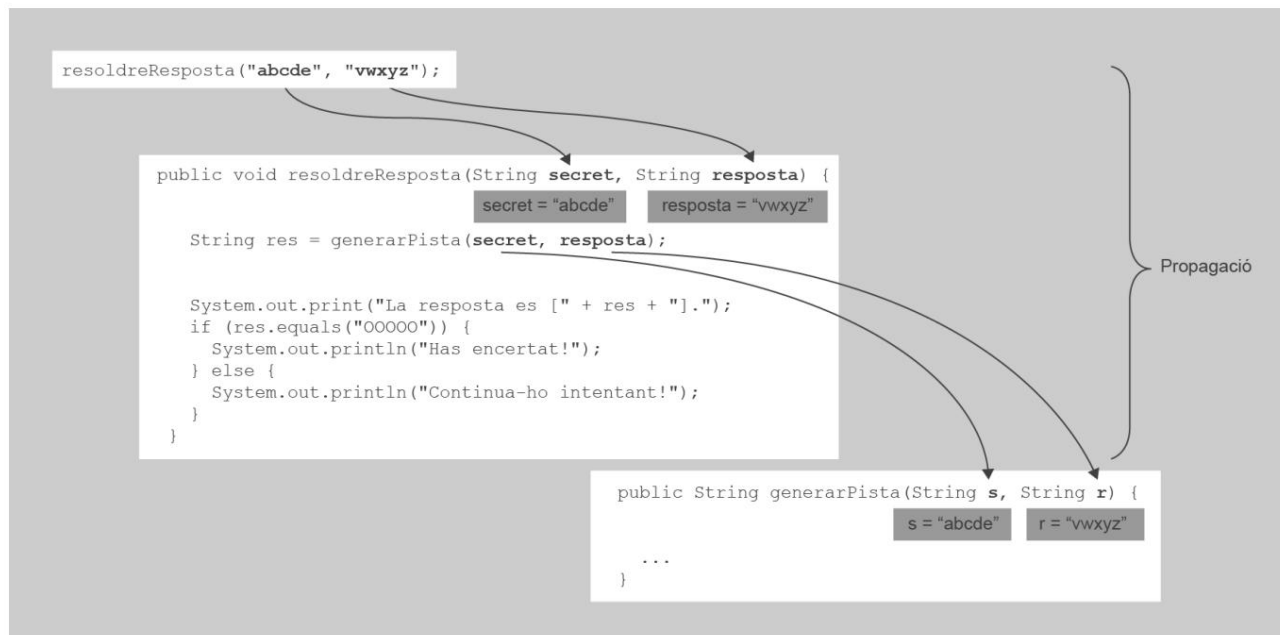
De entrada sólo muestra cosas por pantalla, no calcula nada, por lo que no es necesario ningún valor de retorno. Ahora bien, para poder invocar correctamente generar pista, necesita disponer de los valores del texto secreto y la respuesta del usuario. Estos dos valores no los genera él, sino que son generados por otros métodos. Por tanto, para poder disponer, le harán falta como parámetros de entrada.

```
1 //Parám. entrada: la respuesta y el valor secreto (dos String)
2 //Parám. salida: ninguna (sólo muestra cosas por pantalla) public void
3 resolverRespuesta(String secret, String respuesta) { String nada = generarPista(secret,
4     respuesta); System.out.print("La respuesta se [" + nada + "].");
5     if (res.equals("OOOOOO")) { System.out.println("¡Has acertado!"); } else
6     { System.out.println("¡Sigue
7         intentándolo!");
8
9
10    }
11 }
```

En este método es interesante observar que, internamente, los dos parámetros nunca son usados directamente. Sólo sirven para poder invocar correctamente el método generar pista. Esta situación es la que se denomina propagación de parámetros entre métodos ubicados en distintos niveles de descomposición. Los valores dispuestos en el método de mayor nivel se pasan (propagan) a métodos de nivel inferior. Dado que un parámetro de entrada se considera como una variable cualquiera dentro del código de un método, no hay ningún problema en hacerlo.

La figura 2.5 esquematiza este proceso. También es muy importante ver cómo los valores se van copiando sucesivamente a las variables definidas por los parámetros, independientemente de que su nombre vaya variando en cada caso.

Figura 2. 5. Propagación de los valores de los parámetros entre dos métodos



2.4.4 Implementación del problema general

Una vez implementados todos los subproblemas, ha llegado el momento de implementar el código del problema general: el método inicio. Ante todo lo que hará el programa es generar el valor secreto, y después ir repitiendo la pregunta al usuario en mostrar la respuesta. Esto puede hacerse usando una estructura de repetición controlada por un semáforo, que inicialmente está en false y cambia a true cuando el usuario debe acertado. En una primera aproximación, el código sería:

```

1 //Problema general: Versión 1
2 public void inicio() {
3     String secret = generarPalabraSecreta();
4     //"acertado" dice si se ha acertado y ya se acaba el programa
5     boolean acertado = false;
6     while (!acertado) {
7         String respuesta = preguntarRespuesta();
8         resolverRespuesta(secret, respuesta);
9     }
10 }

```

Sin embargo, en este caso, el problema general también presenta una situación interesante.

Sin variables globales, cómo establecer el valor de la variable acertado en ¿true? Dado que es mejor no usar variables globales, la respuesta debe estar por fuerza en el parámetro de entrada o salida de algún método. En un caso como éste, donde un método necesita un valor para poder llevar a cabo su tarea, la respuesta exacta está al añadir un parámetro de salida a algún método.

Si observa el código, el método que establece si la partida ha terminado (ha acertado o no) es resolverRespuesta, ya que es quien compara la pista con el valor asociado a acertar la palabra secreta. De entrada, este método no tiene ningún valor de retorno. Lo que se puede hacer es añadir un valor de retorno que le diga si es necesario acabar la partida o no (un booleano). De este modo, el código sería:

```
1 //Problema general: Versión 2
2 public void inicio() {
3     String secret = generarPalabraSecreta();
4     boolean acertado = false;
5     while (!acertado) {
6         String respuesta = preguntarRespuesta();
7         acertado = resolverRespuesta(secret, respuesta);
8     }
9 }
10 //Este método corresponde al primer nivel de descomposición
11 //Versión 2
12 //Parám. entrada: la respuesta y el valor secreto (dos String)
13 //Parám. salida: si se ha acertado (un booleano)
14 public boolean resolverRespuesta(String secret, String respuesta) {
15     String nada = generarPista(secret, respuesta);
16     boolean fin = false;
17     System.out.print("La respuesta es [" + nada + "].");
18     if (res.equals(ACIERTAD)) {
19         System.out.println("¡Has acertado!");
20         fin = true;
21     } else {
22         System.out.println("¡Sigue intentándolo!");
23     }
24     return fin;
25 }
```

Éste es un caso donde hay que establecer parámetros de un método a posteriori, frente a una necesidad imprevista de un método de nivel superior. Tampoco se trata de una situación infrecuente.

El proceso de descomposición en nivel descendente no siempre es posible realizarlo linealmente. A menudo es necesario retroceder y reconsiderar decisiones tomadas anteriormente.

2.4.5 Implementación final

Por claridad, a continuación se lista el resultado final. Para hacer el código más pulido y legible, algunos valores concretos se han reemplazado por constantes (la longitud del texto a descubrir y el texto que indica que todo es correcto).

```
1 importe java.util.Scanner;
2 //Un programa para jugar en el Mastermind.
3 public class MasterMind {
4     //Constantes
5     public final static char TOT_CORRECTO = 'O';
6     public final static char MALA_POSICION = 'X';
7     public final static char INCORRECTO = '-';
8     public final static String ACERTADO = "OOOOOO";
9     public final int LONG_SECRET = 5;
10    public final static String abc = "abcdefghijklmnopqrstuvwxyz";
11 //Métodos asociados al problema general
12 public static void main (String[] args) {
13     MasterMind programa = new MasterMind();
14     programa.inicio();
15 }
16 public void inicio() {
17     String secret = generarPalabraSecreta();
```

```
18     boolean acertado = false;
19     while (!acertado) {
20         String respuesta = preguntarRespuesta();
21         acertado = resolverRespuesta(secreto, respuesta);
22     }
23 }
24 //Métodos asociados al primer nivel de descomposición
25 //Parám. entrada: ninguna, todo se obtiene de generarLetraAleatoria
26 //Parám. salida: una palabra de 5 letras (un String)
27 public String generarPalabraSecreta() {
28     String nada = "";
29     for (int i = 0; i < LONG_SECRET; i++) {
30         nada = nada + generarLetraAleatoria();
31     }
32     return nada;
33 }
34 //Parám. entrada: ninguna, todo se obtiene del teclado
35 //Parám. salida: la palabra de respuesta (un String)
36 public String preguntarRespuesta() {
37     Scanner lector = new Scanner(System.in);
38     boolean lecturaOk = false;
39     String nada = null;
40     do {
41         System.out.print("Escribir" nada =      + LONG_SECRET +      " letras minúsculas: ");
42         lector.next();
43         lector.nextLine();
44         lecturaOk = comprobarRespuesta(nada);
45         if (!lecturaOk) {
46             System.out.println("¡Esta respuesta no es válida!");
47         }
48     } while (!lecturaOk);
49     return nada;
50 }
51 //Parám. entrada: la respuesta y el valor secreto (dos String)
52 //Parám. salida: si se ha acertado (un booleano)
53 public boolean resolverRespuesta(String secret, String respuesta) {
54     String nada = generarPista(secreto, respuesta);
55     boolean fin = false;
56     System.out.print("La respuesta es [" + nada + "].");
57     if (res.equals(ACIERTAD)) {
58         System.out.println("¡Has acertado!");
59         fin = true;
60     } else {
61         System.out.println("¡Sigue intentándolo!");
62     }
63     return fin;
64 }
65 //Métodos asociados al segundo nivel de descomposición
66 //Parám. entrada: ninguna
67 //Parám. salida: una letra (un carácter)
68 public char generarLetraAleatoria() {
69     long nano = System.nanoTime();
70     int index = (int)(nano % abc.length());
71     return abc.charAt(index);
72 }
73 //Parám. entrada: texto a comprobar
74 //Parám. salida: si es correcto o no (un booleano. true = correcto)
75 public boolean comprobarRespuesta(String respuesta) {
76     if (respuesta.length() != LONG_SECRET) {
77         //Ya sabemos que no es correcto.
78         //Podemos terminar la ejecución del método inmediatamente.
79         return false;
80     }
81     for (int i = 0; i < respuesta.length(); i++) {
82         char c = respuesta.charAt(i);
83         if ( -1 == abc.indexOf(c)) {
84             //Ya sabemos que no es correcto.
85             //Podemos terminar la ejecución del método inmediatamente.
86             return false;
87         }
88     }
89 }
```

```
88     }
89     //Si todo se cumple, seguro que es correcto
90     return true;
91 }
92 //Parám. entrada: la respuesta y el secreto a comparar (texto)
93 //Parám. salida: la pista a mostrar (un texto)
94 public String generarPista(String s, String r) {
95     String nada = "";
96     for (int i = 0; i < s.length(); i++) {
97         char charSecret = s.charAt(i);
98         char charRespuesta = r.charAt(i);
99         if (charSecret == charRespuesta) {
100             nada = nada + TODO_CORRECTO;
101         } else if (s.indexOf(charRespuesta) != -1) {
102             nada = nada + MALA_POSICION;
103         } else {
104             nada = nada + INCORRECTO;
105         }
106     }
107     return nada;
108 }
109 }
```

2.5 Soluciones a los retos propuestos

Reto 1

```
1 public class EscribirEstrellas {
2     public static void main (String[] args) {
3         Escribir Estrellas programa = new Escribir Estrellas();
4         programa.inicio();
5     }
6     public void inicio() {
7         estrellar(4);
8         estrellar(10);
9         estrellar(20);
10    }
11    //Tiene un parámetro de entrada, de tipo entero
12    public void estrellar(int a) {
13        for(int i = 0; i < a; i++) {
14            System.out.print("**");
15        }
16        System.out.println();
17    }
18 }
```

Reto 2

```
1 //Modificamos el valor de un parámetro. ¿Afecta a la variable original?
2 public class ModificarParametroString {
3     public static void main (String[] args) {
4         ModificarParametroString programa = new ModificarParametroString();
5         programa.inicio();
6     }
7     public void inicio() {
8         String e = "Hola";
9         System.out.println("Antes de llamar al método \"y\" vale " + e);
10
11        System.out.println("Después de llamar al método \"y\" vale " + e);
12    }
13    //Tiene un único parámetro de entrada, de tipo entero
14    public void modificarParametro(String a) {
```

```
15 //Ahora hay una variable "a" declarada.
16 //Su valor depende de cómo se ha invocado el método.
17 a = "Adiós";
18 System.out.println("Hemos modificado el valor en " + a);
19 }
20 }
```

Reto 3

```
1 importe java.util.Scanner;
2 public class LeerReales {
3     public static void main (String[] args) {
4         LeerReals programa = new LeerReals();
5         programa.inicio();
6     }
7     public void inicio() {
8         System.out.println("Leemos un real para el teclado:");
9         double a = leerRealTeclado();
10        System.out.println("Lo real ha sido " + a + ".");
11        System.out.println("Leemos otro real para el teclado:");
12        a = leerRealTeclado();
13        System.out.println("El otro real ha sido " + a + ".");
14    }
15    public double leerRealTeclado() {
16        Scanner lector = new Scanner(System.in);
17        double realLeído = 0;
18        boolean leído = false;
19        while (!leído) {
20            leído = lector.hasNextDouble();
21            if (leído) {
22                realLeído = lector.nextDouble();
23            } else {
24                System.out.println("Esto no es un entero.");
25                lector.next();
26            }
27        }
28        lector.nextLine();
29        return realLeído;
30    }
31 }
```

Reto 4

```
1 public class MultipleMaxim {
2     public static void main (String[] args) {
3         MultipleMaxim programa = new MultipleMaxim();
4         programa.inicio();
5     }
6     public void inicio() {
7         int primeroMinimo = minimo(3, 6);
8         int segundoMinimo = minimo(10, 15);
9         int resMinim = minim(primerMinim, segundoMinim);
10        System.out.println("Lo mínimo es " + resMinim);
11    }
12    //Parám. entrada: los valores a tratar son dos enteros
13    //Parám. salida: el mínimo entre los dos, un entero
14    public int minim (int a, int b) {
15        // "a" y "b" contienen los valores a tratar
16        int min = b;
17        if (a < b) {
18            min = a;
19        }
20        // "min" contiene el resultado
21        return min;
22    }
23 }
```

Reto 5

```
1 public class NotaFinal {
2     public static final double[] notas = {5.0, 7.5, 4.5, 3.75, 8.25};
3     public static void main (String[] args) {
4         NotaFinal programa = new NotaFinal();
5         programa.inicio();
6     }
7     public void inicio() {
8         double res = media(notas);
9         String notaFinal = notaAText(nada);
10        System.out.println("La nota final es " + notaFinal);
11    }
12    //Parám. entrada: el valor a procesar es un array de reales
13    //Parám. salida: el resultado de realizar el cálculo, un real (tiene decimales)
14    public double media(double[] array) {
15        //"array" contiene el valor a tratar
16        double acumulador = 0;
17        for (int i = 0; i < array.length; i++) {
18            acumulador = acumulador + array[i];
19        }
20        //El resultado es (acumulador)/(número elementos)
21        //Se puede devolver el resultado de una expresión
22        return acumulador/array.length;
23    }
24    //Parám. entrada: el valor a procesar es un real (una nota)
25    //Parám. salida: una cadena de texto
26    public String notaAText(double nota) {
27        //"nota" contiene el valor a tratar
28        String text = null;
29        if ((nota >= 9)&&(nota <= 10)) {
30            texto = "Excelente";
31        } else if ((nota >= 6.5)&&(nota < 9)) {
32            texto = "Notable";
33        } else if ((nota >= 5)&&(nota < 6.5)) {
34            texto = "Aprobado";
35        } else if ((nota >= 0)&&(nota < 5)) {
36            texto = "Suspendido";
37        } else {
38            texto = "Nota no válida";
39        }
40        //"texto" contiene el resultado
41        return texto;
42    }
43 }
```

Reto 6

```
1 import java.util.Scanner;
2 public class CuentaValores {
3     public static void main (String[] args) {
4         ComptaValors programa = new ComptaValors();
5         programa.inicio();
6     }
7     public void inicio() {
8         int[] valores = {1, 3, 7, 8, 5, 1, 3, 1, 5, 4};
9         System.out.println("Introduce un número entero:");
10        int valor = leerEnterTeclado();
11        int res = cuentaValores(valores, valor);
12        System.out.println("El valor " + valor + " aparece " + nada + " golpes.");
13    }
14    public int leerEnterTeclado() {
15        Scanner lector = new Scanner(System.in);
16        int enteroLeído = 0;
17        boolean leído = false;
18        while (!leído) {
19            leído = lector.hasNextInt();
20            if (leído) {
21                enterLeído = lector.nextInt();
```

```
22         } else {  
23             System.out.println("Esto no es un entero.");  
24             lector.next();  
25         }  
26     }  
27     lector.nextLine();  
28     return enteroLeído;  
29 }  
30 public int cuentaValores(int[] array, int valor) {  
31     int acumulador = 0;  
32     for (int i = 0; i < array.length; i++) {  
33         if (array[i] == valor) {  
34             acumulador++;  
35         }  
36     }  
37     return acumulador;  
38 }  
39 }
```